

CURS 1

Bibliografie

1. Sabin Buraga si Gabriel Ciobanu –“Atelier de programare in retele de calculatoare”; Ed. Polirom
2. Andrew Taneubaum – “Modern operating systems”
3. Ioan Jurca – “Sisteme de operare”, Ed. Vest
4. Dragos Apostachioaie – “Securitatea sistemelor Linux”, Ed. Polirom

Def. Un sistem de operare = set de programe care asigura interfata utilizatorului cu sist de calcul **extinzand si simplificand** setul de operatii disponibile; asigura gestionarea resurselor:

- fizice (proc,mem,adr. Periferice)
- logice (procese, fisiere, proceduri)

Fisierul = unitatea de baza pentru stocarea informației

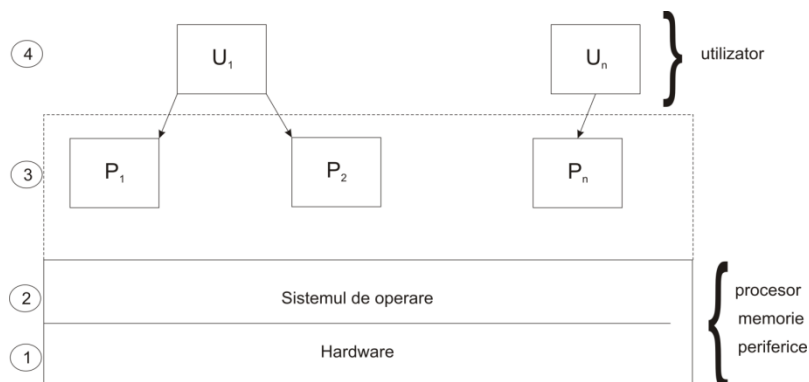
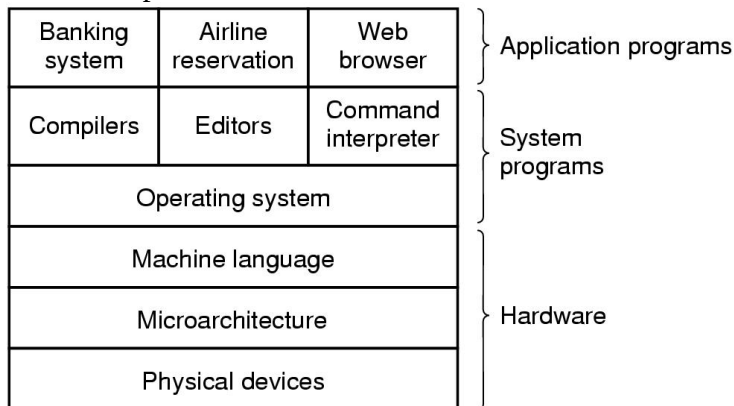
Procesul = o aplicatie in stare de execuție

Componentele unui sistem de calcul:

- hardware-ul : furnizează resurse precum: procesor, memorie, periferice.
- sistemul de operare : pune la dispoziția utilizatorului si a programelor resursele hardware.

Sistemele de operare au avut o evoluție continua de la simplul suport pentru execuție catre sisteme complexe de gestionare a resurselor care ușurează sarcinile utilizatorului.

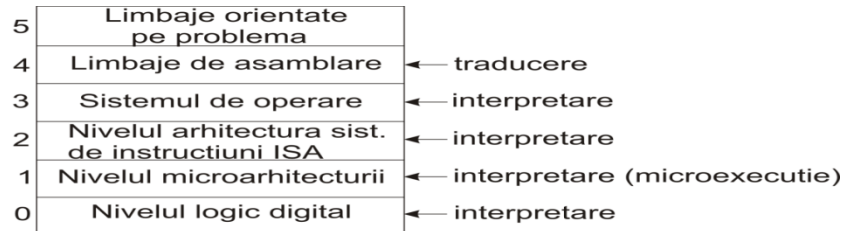
- programe de aplicație: pe baza sistemului de operare rulează programe de aplicație. Programele de aplicație definesc căile prin care resursele sistemului sunt utilizate pentru rezolvarea problemelor de calcul ale utilizatorilor.



- utilizatorii: pot fi persoane, masini sau alte calculatoare

Organizarea unui calculator

Tanenbaum prezintă calculatorul ca fiind o structura organizata pe mai multe niveluri. Trecerea de la un nivel la altul se face prin interpretare sau traducere



Limbajele de programare – sunt de două tipuri:

- de nivel înalt (independente de mașina pe care se procesează – microarhitectura hardware) • C • Fortran – rezolvarea calculelor științifice • Cobol – rezolvarea problemelor de procesare a datelor specifice afacerilor (aplicații economice) • Prolog – folosit în aplicații care necesită proiectarea unui sistem expert.

-de nivel scăzut (low – dependente de mașina pe care se procesează).

Există câte un astfel de limbaj specific fiecărei familii de procesoare.

• *Limbajul de asamblare.*

Interpretarea înseamnă ca un program este executat pas cu pas. Fiecare instrucțiune a pasului L_i fiind o data de intrare pentru sistemul L_{i+1}

Traducerea: Fiecare instrucțiune de la nivelul L_i este înlocuita cu o noua secvența de cod echivalentă la nivelul L_{i+1} . Traducerea unui program de nivel înalt în ISA aferent calculatorului

Arhitectura Setului de Instrucțiuni (ISA) – reprezintă interfața dintre software (programele de aplicație / sistem de operare) și hardware-ul care îl execută. Nivelul arhitecturii sistemului de instrucțiuni ISA (nivelul 2) realizează o interpretare parțială.

Microarhitectura – organizarea detaliată a unei implementări de ISA. Microarhitectura pune la dispoziția nivelului superior funcții de bază precum selecția unei locații de memorie, activarea scrierii, activarea citirii.

SARCINILE SISTEMULUI DE OPERARE

A. Asigurarea interfetei cu utilizatorul

Reprezintă orice instrument care permite preluarea comenzilor de la utilizator. În majoritatea cazurilor este vorba de interfața software.

Sunt trei tipuri de interfețe:

1. Monitoare – sistemele de operare vechi au stocat în memoria ROM a calculatorului un set minimal de funcții numit *program monitor* care permitea inspectarea registrilor, vizualizarea conținutului memoriei, etc.

2. Interfața în linie de comandă – reprezentată de un program numit interpretor de comenzi care afișează un prompter, primește comanda și o execută. Toate sistemele de operare au un interpretor de comenzi, mai complex (UNIX) sau mai simplu (WINDOWS);
3. Interfața grafică – prezintă un set de obiecte prin care utilizatorul poate comunica cu sistemul de operare.

Comparatie între interfața text și interfața grafică

Interfața grafică:

- avantaje: este intuitivă, poate fi învățată mai ușor, permite utilizarea aplicațiilor complexe;
- dezavantaje: utilizează resurse, ascunde detalii de prelucrare.

Interfața text:

- avantaje: comunicarea este simplă și eficientă, permite definirea clară a comenzilor, resurse minime de calcul.

B. Gestionarea Procese și procesoare

C. Gestionarea memoriei

Memoria este utilizată de sistemul de operare dar și de procesele care pot lucra în paralel. Modul în care este alocată și împartită memoria și cooperarea între SO și procese implică modulul de gestionare a memoriei.

Mecanismele de swapping și memorie virtuală permit utilizarea memoriilor externe ca extensii a memoriei interne.

D. Gestionarea perifericelor

Cuprinde aspectele operațiilor de introducere și extragere a datelor: pregătirea operației, lansare a cererilor și controlul transferurilor.

E. Gestionarea fișierelor

F. Tratarea erorilor

Înseamnă detectarea erorii, tratarea acesteia și modul de revenire la executia normală

Caracteristicile SO

1. Modul de introducere a programelor în sistem
 - sisteme de operare seriale (datele sunt introduse de la un singur dispozitiv de intrare);
 - sisteme de operare paralele (datele sunt introduse de la mai multe dispozitive de intrare);
 - sisteme de operare cu introducere de la distanță;
2. Modul de planificare a lucrărilor pentru execuție:
 - SO orientate pe lucrare: admit ca unitate de planificare o lucrare alcătuită din unul sau mai multe programe;
 - SO orientate pe proces: ca unitate de planificare admit procesul;
3. Numărul de programe prezente simultan în memorie
 - monoprogramare
 - multiprogramare (mai multe programe simultan în memorie)
4. Gradul de comunicare a proceselor în multiprogramare
 - monotasking: programele din memorie nu au obiective comune și nu pot comunica între ele;
 - multitasking (WINDOWS): programele pot comunica între ele;

5. Numarul de utilizatori simultani
 - monouser(MS-DOS);
 - multiuser (WINDOWS);
6. Modul de organizare a resurselor
 - SO cu resurse alocate : alocarea se face pe toata durata executiei
 - SO in timp real: permit controlul executarii proceselor intr-un interval de timp specificat (de exemplu in conducerea unui proces industrial, procesul trimite SO parametrii, iar sistemul de operare trimite procesului deciziile luate);
 - SO cu resurse partajate: accesul la resurse poate fi direct (pentru SO in timp real), multiplu, time sharing, si la distanta.

Componentele sistemelor de operare se grupeaza pe nivel fizic (aproprate de hardware), interacționând cu acesta prin întreruperi. Întreruperea este oprirea executiei normale, la aparitia unui eveniment.

Pe nivel logic, mai apropiat de utilizator, comunicând cu acesta prin comenzi si limbaje de programare.

Programele executate de sistemele de operare se pot prin urmare grupa in doua categorii:

- programele de control si comanda : care realizeaza interfatarea cu nivelul hardware;
- programe de servicii: executate sub supravegherea programelor de control utilizate de utilizator.

Un sistem de operare are doua aspecte structurale: kernel si user.

Kernelul realizeaza procese de baza (de ex. gestionarea proceselor, mecanisme de sincronizare a proceselor si gestionarea memoriei si a perifericelor).

Modul Utilizator ruleaza aplicatiile.

CURS 2

Planificarea procesoarelor

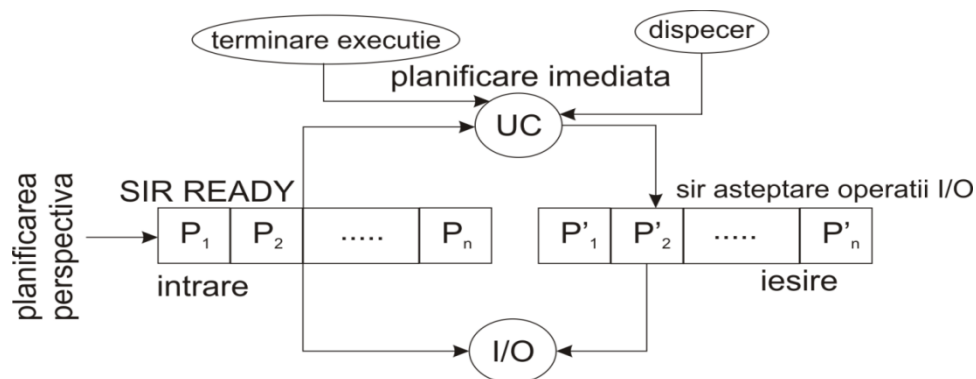
Scopul este utilizarea cat mai buna a procesoarelor. Multiprogramarea este un concept folosit in S.O. care se datoreaza existentei in memorie a mai multor procese care trebuie planificate pentru a putea utiliza unitatea centrala cat mai eficient. Procesele pot fi limitate de 2 elemente:

- Unitatea centrala: cand procesul are timp majoritar de desfasurare in operatii de I/O.
- Operatiile de intrare-iesire daca exista ocuparea majoritara a U.C.

In memorie procesele stau grupate intr-un sir de asteptare pentru a le fi alocata U.C. Implementarea acestui sir este o structura coada si anume o lista inlantuita.

Componentele schemei de planificare a procesoarelor

1. Planificatorul pe termen lung: care stabileste ce procese sunt incarcate in memorie; are o frecventa mica; el controleaza gradul de multiprogramare;
2. Dispecerul: acesta are rolul de a aloca U.C. proceselor si are o frecventa mare si trebuie sa fie foarte rapid ;
3. Planificatorul pe termen scurt: acesta preia procesele incarcate in memorie de planificatorul pe termen lung si il aloca U.C.



Criterii de performanta in planificarea unitatii centrale

1. Gradul de utilizare al unitatii centrale: scopul este utilizarea cat mai buna si sunt 2 niveluri sub 40% sistem cu incarcare redusa peste 90% sistem cu incarcare mare
2. Numarul de procese executate intr-un interval de timp: Throughput
3. Turnaround time: reprezinta durata totala a unui proces, fiind timpul de la incarcarea procesului in memorie si pana la terminarea executiei
4. Durata de asteptare: se refera la timpul petrecut in sirul READY (se calculeaza doar timpul de asteptare din sirul ready)
5. Durata de raspuns: timpul de la formularea unei cereri pana la timpul de raspuns al acesteia
6. Corectitudinea privind alocarea unitatii centrale in mod egal pentru fiecare dintre ele (threadul(timp de executie) este cea mai mica entitate care poate fi planificata de un sistem de operare). Threadul reprezinta bifurcatia unui proces in 2 sau mai multe sarcini concurente. Threadul este o subunitate a unui proces.

ALGORITMI DE PLANIFICARE AI UNITATII CENTRALE

Bibliografie: <http://www.personal.kent.edu/~rmuhamma/OpSystems/os.html>

Deși sunt numeroase caracteristici care pot fi urmărite, valoarea folosită pentru comparație este DMA (durata medie de așteptare).

Algoritmul FIFO(first come first serv)

Primul proces care intră în sirul READY este primul executat.

Proces	Timp execuție	Durata așteptare
P1	12	0
P2	3	12
P3	4	3

Durata medie de așteptare: $DMA = \frac{0 + 12 + 3}{3} = 5$

Un procesor împarte execuția în secvențe de timp cu durată între **10 și 100 de milisecunde**. Timpul de execuție marchează câte astfel de secvențe de timp sunt necesare.

Se observă că durata medie de așteptare nu este minimă și depinde de ordinea și complexitatea proceselor.

Algoritmul Shortest Job First (SJF)

Cea mai scurtă sarcină din sirul READY este prioritară; în caz de egalitate se execută algoritmul anterior.

Proces	Timp execuție	Durata așteptare
P1	12	12
P2	2	0
P3	2	2
P4	3	4
P5	5	7

Ordinile : P2(2); P3(2), P4(3), P5(5), P1(12)

Dezavantajul este că procesele cu timp de execuție mare pot să aștepte foarte mult (la infinit).

Durata medie de așteptare: $DMA = \frac{2 + 4 + 7 + 12}{5} = 5$

Acest algoritm este optimal dacă se cunoaște durată de execuție a U.C. Procesele intrate în execuție nu sunt întrerupte.

Algoritmi bazati pe prioritate

Fiecarui proces *i* se alocă o prioritate; U.C. este alocată procesului cu prioritate maximă.

Există două tipuri de priorități:

- Interne (masurabile; ex: limita de timp, cantitatea de memorie, nr fișierelor deschise și raportul între operațiile i/o și cele cu U.C.);
- Externe (decizia utilizatorului, factori financiari, tip departament).

Priority scheduling can be either preemptive or non preemptive

- A preemptive priority algorithm will preempt the CPU if the priority of the newly arrival process is higher than the priority of the currently running process.
- A non-preemptive priority algorithm will simply put the new process at the head of the ready queue.

Aici intervine posibilitatea blocarii la infinit sau infometarea proceselor (ele sunt intrerupte la infinit de procese prioritare). Solutia este imbatranirea proceselor adica marirea prioritatii odata cu trecerea timpului.

Algoritmi preemptivi

Ruperea executiei unui proces in momentul alocarii la unitatea centrala. Primul algoritm prezentat (FCFS) nu este preemptiv; SJF poate fi realizat preemptiv. Algoritmii bazati pe prioritate sunt realizati preemptiv.

Trei procese sosesc in sirul READY la momentele de timp indicate in figura:

Proces	Moment sosire	Timp executie
P1	0	12
P2	3	3
P3	4	6

SJF nepreemptiv	SJF preemptiv
P1 durata asteptare= 0	P1 durata de asteptare=9
P2 durata asteptare=9	P2 durata asteptare=0
P3 durata asteptare=8 de la P1 + 3 de la P2	P3 durata de asteptare=2

$$DMA_{nepreemptiv} = \frac{20}{3} = 6,3$$

$$DMA_{preemptiv} = \frac{11}{3} = 3,6$$

Algoritmul Round Robin

Algoritmul utilizeaza functia U.C. de a executa operatiile in cuante de timp (intre 10 ms si 100 ms). Algoritmul este de tip time-sharing (partajare timp), fiecarui proces fiindu-i alocata U.C. pentru un numar de cicluri. Sirul READY este tratat ca un FIFO circular.

Ex: Presupunem ca un proces poate fi alocat maxim 10 cuante de timp.

Proces	Timp executie
P1	29
P2	16
P3	15
P4	3

Ordinea executie este: P1(19), P2(6), P3(5) P4(0), P1(9), P3(0), P1(0)

Utilizarea procesorului este foarte mare. Marimea ciclului afecteaza performantele algoritmului Round Robin. Daca ciclul este foarte mare se comporta ca algoritmul FCSF. Daca ciclul se micsoreaza, performanta scade datorita opratiilor de comutare a regiștrilor.

Timpul de asteptare este aproape egal pentru toate procesele.

Earliest Deadline First Scheduling (EDFS)

algoritm preemptiv de planificare dinamică

nu impune ca procesele să fie periodice sau să utilizeze aceeași durată de timp CPU la fiecare execuție

Când un proces are nevoie de timp CPU, își anunță intenția, specificând și termenul limită.

Planificatorul menține o listă a proceselor "gata de execuție", sortată pe baza termenelor limită.

Algoritmul de planificare va executa procesul din listă cu cel mai apropiat termen limită.

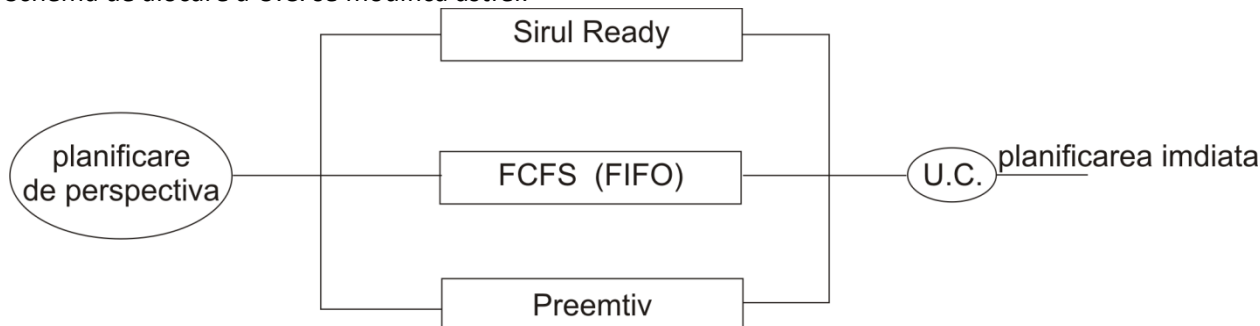
Ori de câte ori un proces trece în starea "gata de execuție" se verifică dacă termenul limită al acestuia este mai mic decât termenul limită al procesului în execuție.

În caz afirmativ, noul proces preemtează procesul în execuție.

Algoritmul EDFs lucrează corect pentru orice set planificabil de procese și poate atinge un grad de utilizare a procesorului de 100%.

Alți algoritmi de planificare multi nivel

Sistemul de operare Windows XP folosește algoritmi cu siruri de procesare multinivel. El combina algoritmi preemptivi Round-Robin și FIFO. Procesele își cresc prioritatea în funcție de timpul de așteptare. Schema de alocare a U.C. se modifică astfel:



Very early MS-DOS and Microsoft Windows systems were non-multitasking, and as such did not feature a scheduler. Windows 3.1x used a non-preemptive scheduler, meaning that it did not interrupt programs. It relied on the program to end or tell the OS that it didn't need the processor so that it could move on to another process. This is usually called cooperative multitasking. Windows 95 introduced a rudimentary preemptive scheduler; however, for legacy support opted to let 16 bit applications run without preemption.

Windows NT-based operating systems use a multilevel feedback queue. 32 priority levels are defined, 0 through to 31, with priorities 0 through 15 being "normal" priorities and priorities 16 through 31 being soft real-time priorities, requiring privileges to assign. 0 is reserved for the Operating System. Users can select 5 of these priorities to assign to a running application from the Task Manager application, or through thread management APIs. The kernel may change the priority level of a thread depending on its I/O and CPU usage and whether it is interactive (i.e. accepts and responds to input from humans), raising the priority of interactive and I/O bounded processes and lowering that of CPU bound processes, to increase the responsiveness of interactive applications.^[9] The scheduler was modified in Windows Vista to use the cycle counter register of modern processors to keep track of exactly how many CPU cycles a thread has executed, rather than just using an interval-timer interrupt routine. Vista also uses a priority scheduler for the I/O queue so that disk defragmenters and other such programs do not interfere with foreground operations.

Sistemul de operare Linux kernelul 2.4 folosește tot algoritmi multi-nivel. El are 140 de nivele de prioritate din care o parte sunt alocate intreruperilor. 0–99 are reserved for real-time tasks and 100–140 are

considered nice task levels. For real-time tasks, the time quantum for switching processes was approximately 200 ms, and for nice tasks approximately 10 ms.

In contrast to the previous scheduler used in older Linux 2.6 kernels, the CFS scheduler implementation is not based on run queues. Instead, a red-black tree implements a "timeline" of future task execution. Additionally, the scheduler uses nanosecond granularity accounting, the atomic units by which an individual process' share of the CPU was allocated (thus making redundant the previous notion of timeslices)

Tabel comparativ pentru cei 5 algoritmi:

Algoritm	Utilizare CPU	Throughput	Turn around time	Tratare dealine	Infometarea procesor
FCFS	Scazut	Scazut	Mare	NU	NU
SJF	Mediu	Mediu *	Mediu	NU	DA
Algoritmi cu prioritate	Medie	Scazut	Mare	DA	DA
Preemptiv	Mare	Mediu	Mediu	NU	DA
Multi-nivel	Mare	Mare	Mediu	DA	DA

*Algoritmul SJF poate avea un throughput mare in functie de durata de executie a U.C.

A multilevel queue scheduling algorithm partitions the ready queue in several separate queues, for instance. In a multilevel queue scheduling processes are permanently assigned to one queues.

The processes are permanently assigned to one another, based on some property of the process, such as

- Memory size
- Process priority
- Process type

Algorithm choose the process from the occupied queue that has the highest priority, and run that process either

- Preemptive or
- Non-preemptively

Each queue has its own scheduling algorithm or policy.

Possibility I

If each queue has absolute priority over lower-priority queues then no process in the queue could run unless the queue for the highest-priority processes were all empty.

For example, in the above figure no process in the batch queue could run unless the queues for system processes, interactive processes, and interactive editing processes will all empty.

Possibility II

If there is a time slice between the queues then each queue gets a certain amount of CPU times, which it can then schedule among the processes in its queue. For instance;

- 80% of the CPU time to foreground queue using RR.
- 20% of the CPU time to background queue using FCFS.

Since processes do not move between queue so, this policy has the advantage of low scheduling overhead, but it is inflexible.

CURS 3

Gestiunea proceselor

Procesele sunt entitățile active ale S.O. Diferența între proces și program este aceea că programul este o entitate pasivă. De exemplu un algoritm reprezintă un program, iar un proces este reprezentat de execuția algoritmului. Uneori procesele mai poartă denumirea și de task. Spațiul de adresă al unui proces cuprinde 3 elemente:

- **Segmentul de cod:** care conține imaginea executabilă a programului;
- **Segmentul de date:** el este unic pentru fiecare proces și conține zonele de date, zona de rezervări și zona de alocare dinamică;
- **Segmentul de stivă:** care nu este partajat;

Pentru ca procesul blocat să poată relua execuția din momentul în care a fost întrerupt este necesară salvarea mai multor informații. Structura care conține aceste date se numește PCB (Process Control Block) sau traducerea BCP (Blocul de Control al Proceselor). El conține:

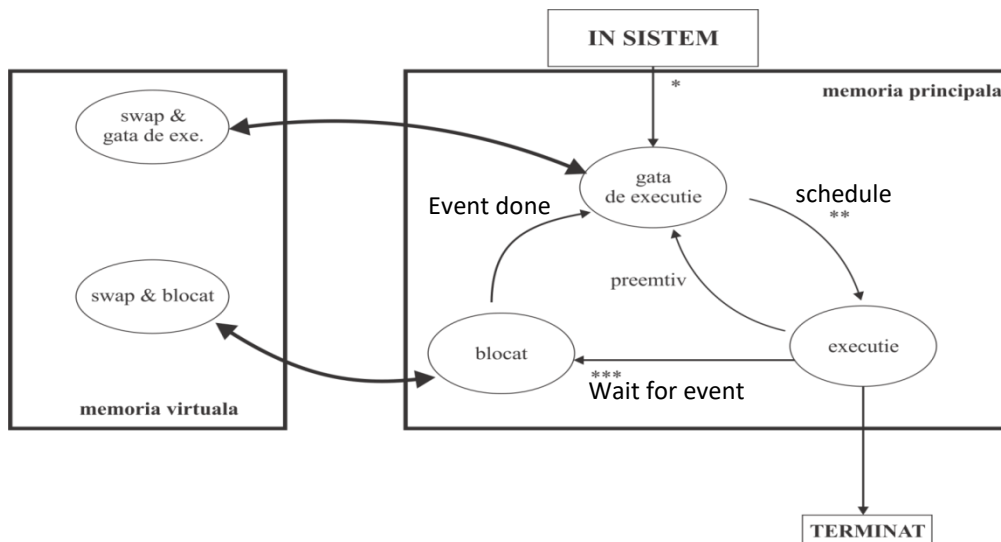
- PID-ul
- PC-ul (program counter): *is a processor register that indicates where a computer is in its program sequence. In most processors, the PC is incremented after fetching an instruction, and holds the memory address of ("points to") the next instruction that would be executed. (In a processor where the incrementation precedes the fetch, the PC points to the current instruction being executed.) Instructions are usually fetched sequentially from memory, but control transfer instructions change the sequence by placing a new value in the PC. These include branches (sometimes called jumps), subroutine calls, and returns. A transfer that is conditional on the truth of some assertion q lets the computer follow a different sequence under different conditions.*
- Elemente Hardware
 - Stack-pointerul (indica locația ultimului obiect plasat în stivă de procesare)
 - Registrul U.C.*
 - Indicatorul de prioritate
- Zona pentru salvarea stării la prelevarea resursei procesor
- Pointerul spre lista proceselor aflate în aceeași stare

Stările procesului: un proces poate să aibă 3 stări:

- Gata de execuție ;
- Execuție
- Blocat

Acestea 3 dacă ne aflăm în memoria principală și încă 2 stări în memoria virtuală:

- SWAP și gata de execuție
- SWAP și blocat



* Datele sunt aduse in memorie de planificatorul pe termen lung.

Procesele gata de execuție se afla in coada READY si așteaptă eliberarea procesorului pentru a putea fi executate.

** Planificatorul pe termen scurt

Daca algoritmul de planificare al U.C. este preemptiv, procesul se poate reintoarce in coada de asteptare.

*** Exista posibilitatea ca un proces sa aștepte accesul la resursele I/O. Pana la alocarea resursei procesul este in starea blocat. Când resursa este disponibila procesul este gata de executie (reintra in șirul READY). Uneori calculatorul nu are suficiente resurse de memorie si foloseste memoria virtuala (SWAP). Stările ce pot fi trecute in starea SWAP sunt blocate si gata de execuție.

Comutarea proceselor

Tranziția sau comutarea intre procese intr-un sistem multi-tasking are loc la apariția unui eveniment. Comutarea implica un anumit timp si poate influenta performantele sistemului.

Protecții hardware

Sistemul de operare are acces la resurse pe care utilizatorii, daca le-ar accesa prin programele lor ar pune in pericol stabilitatea S.O.

Exemplu: modificare zone de memorie folosite de S.O. ; suprascrierea fisierelor

Prin urmare au fost introduse trei protecții pentru a asigura stabilitatea S.O. :

- Protecția intrarilor si a iesirilor din sistem
- Protecția memoriei
- Protecția procesorului

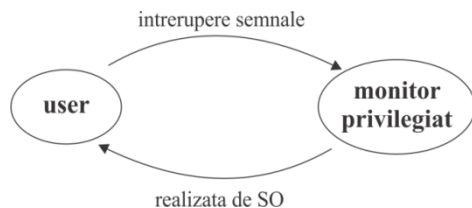
Pentru a putea implementa aceste protecții au fost definite 2 tipuri de instructiuni :

- Instructiuni privilegiate
- Instructiuni neprivilegiate

Corespunzător exista doua moduri de operare ale procesorului:

- Modul user (neprivilegiat)
- Modul monitor (privilegiat)

Trecerea din modul user in modul monitor se face prin întreruperi si semnale. Din modul monitor in user se face trecerea de către S.O.



Daca un proces utilizator încearcă sa execute instrucțiuni privilegiate el este terminat. Singurul care poate executa instrucțiuni privilegiate este S.O. Exista un bit hardware care identifica instrucțiunile in modul privilegiat de instrucțiunile in modul user (privilegiat).

Cererile adresate de utilizator către S.O. pentru a executa instrucțiuni privilegiate se numesc *systemcalls*.

O apelare a acestei cereri duce la apariția unei întreruperi sau a unui semnal, iar procesorul este comutat in rolul privilegiat.

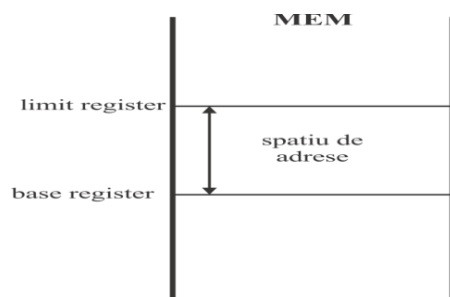
Detaliere protecții pentru a asigura stabilitatea SO:

*Primul element este **protectia intrarilor si a iesirilor**, previne de exemplu blocarea memoriei sau blocarea accesului la procesor. Prin urmare toate instructiunile care acceseaza sistemul I/O sunt privilegiate.

***Protectia memoriei**: zona de memorie folosita de un proces si se numeste **spatiu de adrese al procesului**. Fiecare proces este insotit de doua limitatoare:

- Base register: Base register defineste adresa cea mai mica a procesului
- Limiter register: stabileste dimensiunea procesului

La fiecare executie a unui proces S.O. verifica cele 2 valori.



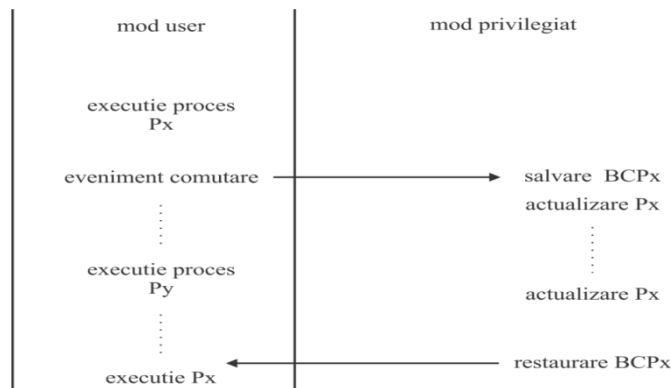
Exista aplicatii care pot fi folosite pentru a urmarii executia proceselor in memorie.

***Protectia procesorului** se realizează prin alocarea unui timer pentru fiecare proces. Perioada este determinata prin folosirea unui ceas fix si a unui registru care este decrementat. Când registrul atinge valoarea 0 se întrerupe accesul procesului la CPU si se executa următorul proces din sirul READY. Valoarea timerului este variabila.

During [context switch](#), the running process is stopped and another process is given a chance to run. The kernel must stop the execution of the running process, copy out the values in hardware registers to its PCB, and update the hardware registers with the values from the PCB of the new process.

Since PCB contains the critical information for the process, it must be kept in an area of memory protected from normal user access. In some operating systems the PCB is placed in the beginning of the kernel [stack](#) of the process since that is a convenient protected location.

Graficul următor simbolizeaza modul in care se realizeaza comutarea proceselor:



Deoarece in cazul threaduri-lor elementul principal care difera este zona de stiva si registrii hardware, insa zona de memorie este aceeaasi; asa poate creste eficienta operatiei de comutare prin folosirea threaduri-lor. Toate resursele in cazul mai multor threaduri sunt gestionate de un singur proces (acelasi proces). Numai este necesara comutarea partilor de memorie.

Crearea si terminarea proceselor

Exista 4 evenimente care determina crearea proceselor:

- Initializarea sistemului
- Executia unui apel de sistem (systemcall) – apelarea produce crearea unui nou proces in modul privilegiat;
- Cerere a utilizatorului
- Initierea unui task in fundal – procesele pot fi de 2 tipuri: in prim-plan sau in fundal. Cele de fundal se mai numesc in Linux: *daemon* iar in Windows *servicii*

Procesele pot fi create in majoritatea cazurilor de catre alte procese. Procesul original se numește părinte; procesul creat se numește fiu; cele 2 procese sunt independente si lucreaza nesincronizat; segmentele de cod executate sunt diferite; activitatea lor poate fi sincronizata.

In Windows spatiile de adrese pentru procesele parinte si copil sunt create distinct de la inceput. In Linux, initial, copilul copiaza intreg spatiul de adrese al parintelui.

Terminarea proceselor poate fi determinata de:

- iesire normala (voluntar);
- iesire cu eroare (voluntar);
- eroare fatala (involuntar- **Ex:** executie instructiune ilegala);
- terminare de catre un alt proces (Ex.: KILL -9 in Linux; iar in Windows: taskkill).

In Linux sunt definite 256 de coduri de eroare si un singur cod de terminare corecta (Return 0). Ele pot fi analizate pentru a determina modul in care s-a realizat executia unui program.

In Linux toate procesele inrudite creaza un grup de procese. Fiecare proces are un singur parinte si poate avea mai multi copii. In Windows nu exista ierarhie. Toate procesele sunt egale.

In Windows entitatea de alocare a timpului procesoarelor este thread-ul; fiecare process contine un thread principal care poate crea thread-uri noi. Un proces Windows are o dualitate de identificare care consta intr-un handle (o intrare in tabelul de resurse al sistemului) si un PID. In concluzie lucrul cu procese in Windows este mai dificil decat in Linux, deoarece unele functii solicita drept parametru handler-ul, iar altele PID-ul.

CURS 4

Comunicarea si sincronizarea intre procese

De multe ori procesele sunt nevoite sa comunice intre ele. Comunicarea trebuie sa fie bine structurata adică sa nu aiba întreruperi.

Conditii de cursa (race condition): procesele care lucreaza impreuna pot partaja o zona de memorie in care fiecare poate citi si scrie.

Ex :coada de tiparie

Exista 2 grupuri de procese care accesează coada de tipărire. Unul dintre ele este aplicația utilizator, iar celălalt este procesul de fundal care preia datele din coada de tipărire si le trimite la imprimanta. Condiția de cursa apare când 2 procese vor sa scrie in zona de date partajata.

Procesele pot fi clasificate:

1. **Procese independente:** daca execuția nu afectează alte procese si nu este afectata de alte procese.

- Pot fi pornite si oprite fără problema
- Sunt deterministe: Sunt reproductibile adică 2 execuții succesive duc la același rezultat
- Nu folosesc date in comun cu alte procese

2. Procese cooperante

***Problema secțiunii critice si a excluderii mutuale:** de multe ori apare problema cand memoria sau fisierele sunt partajate. Trebuie stabilit in aceasta situatie un mod care sa interzica citirea si scrierea simultana a datelor partajate. **Zona de program in care memoria partajata este accesata se numește secțiune (regiune) critica.** Pentru a evita condițiile de cursa trebuie sa ne asiguram ca doua procese nu sunt niciodată simultan in secțiunea critica. Pentru eliminarea erorilor datorate executiei concurente este necesar ca:

- Secventa de instructiuni din care este alcatuita operatia de actualizare a unei variabile partajate sa fie atomica (nu poate fi intrerupta)
- Operatia de actualizare a unei variabile partajate trebuie sa inhibe executia unei operatii similare asupra acelei variabile

Este prin urmare necesara serializarea operațiilor asupra variabilelor partajate. Solutionarea problemei excluderii mutuale trebuie sa îndeplinească urmatoarele cerințe:

1. Oricare doua procese sa nu fie simultan in secțiunea critica;
2. Nu se poate face o presupunere legata de numărul de procesoare si viteza acestora
3. Nici un proces rulând in afara secțiunii critice un poate bloca alte procese
4. Nici un proces nu va aștepta la infinit sa intre in secțiunea lui critica

Un proces urmărește mecanismul prezentat in continuare:

- Negociere asupra accesării resursei (învingătorul isi continua execuția)
- Execuție secțiune critica
- Deconectare (eliberare resursa)

Suportul hardware pentru implementarea excluderii mutuale

1. Dezactivarea întreruperilor

Desi s-au incercat solutii software pentru solutionarea excluderii mutuale, solutiile eficiente se bazeaza pe suport hardware. Instructiunile de invalidare si validare intreruperi (DI si EI) sunt disponibile in orice procesor. Fiecare proces dezactiveaza toate intreruperile la intrarea in regiunea critica si le reactiveaza la iesire.

Procesorul comuta între procese prin intreruperi de ceas sau alte intreruperi. Secvența care asigură accesul exclusiv la resurse urmează apoi aceiași pași DI apoi accesare resurse (exclusiv) și în final EI.

Dezavantajele acestei soluții:

- Se blochează și alte procese (numite “victime inocente”)
- Dacă intreruperile sunt executate de utilizatori pot bloca sistemul, iar dacă sunt executate de kernel implică un cost suplimentar de resurse
- Pe sistemele multiprocesor nu funcționează deoarece dezactivarea intreruperilor se face per procesor

2. Instrucțiunea Test & Set

Instrucțiunea este folosită pentru a scrie într-o locație de memorie și a returna fosta valoare într-o singură operație. Pași de execuție sunt:

- Se compară valoarea operandului cu o valoare constantă și se setează indicatorii de condiție ai procesorului (de exemplu BUSY).
- La finalul execuției și setează operandul la valoarea BUSY

În cod C implementarea este:

```
function lock (bool lock)
{
    while (test_and_set (lock)==1)
}
```

Practic procesul care apelează funcția reușește blocarea resursei doar dacă fosta valoare a lui lock este 0. Dacă aceasta nu este 0 se așteaptă momentul respectiv.

Probleme:

Presupunem 2 procese executate în paralel. La momentul t_1 se testează dacă lock este 0. Procesul 2 testează și el lock=0. Între timp procesul 1 revenit din ce avea de făcut înainte și setează lock=1 și apoi procesul 2 setează și el lock=1 → condiții de cursă.

Soluția ar fi testarea în continuu a variabilei lock (pierdere timp de procesor) sau folosirea unui program extern care să scrie variabila lock și care să poată serializa operația.

Protocoale de așteptare în excluderea mutuală

Cele 2 protocoale sunt BUSY-WAIT (speed lock) și SLEEP-WAIT.

BUSY-WAIT (speed lock)

Acest protocol se bazează pe observația anterioară în care un proces verifică în mod repetat dacă o condiție este adevărată (exemplu: blocarea unei resurse). **Procesul rămâne activ dar nu execută sarcini utile.** Procesele nu sunt terminate decât de utilizator sau de sistemul de operare dacă intră în modul sleep. Implementarea este eficientă dacă blocarea durează foarte puțin deoarece se evită utilizarea de resurse pentru comutarea proceselor. Dacă așteptarea este lungă ea influențează performanța procesorului (procesorul stă blocat executând bucle foarte lungi în care nu se face nimic). Implementarea nu se poate face cu limbaj de nivel înalt pentru că acestea nu au instrucțiuni atomice adevărate. **Implementarea este imposibilă pe procesoarele cu execuție out of order.**

SLEEP-WAIT

În acest protocol **procesul care are resurse disponibile este suspendat și introdus într-o coadă de așteptare. El este trezit când resursa devine disponibilă.**

În sistemele uniprocesor se folosește mereu sleep wait, în cele multiprocesor se folosește o combinație a celor două.

In prezent exista 2 tipuri de procesoare: *in order* si *out of order* (acestea se refera la ordinea executarii operatiilor).

IN ORDER (utilizata la procesoarele Atom) ordinea este:

1. fetch; instructiune
2. asteptare resurse (timppierdut)
3. executie instructiune
4. scriere rezultat

In earlier processors, the processing of instructions is normally done in these steps:

1. **Instruction fetch.**
2. If input **operands** are available (in registers for instance), the instruction is dispatched to the appropriate **functional unit**. If one or more operand is unavailable during the current clock cycle (generally because they are being fetched from **memory**), the processor stalls until they are available.
3. The instruction is executed by the appropriate functional unit.
4. The functional unit writes the results back to the **register file**.

OUT OF ORDER (restul):

1. fetch instructiune
2. ordonare instructiuni in coada de asteptare
3. asteptare resurse
4. executie instructiuni ale caror resurse sunt disponibile
5. salvare rezultate si ordonare

This new paradigm breaks up the processing of instructions into these steps:

1. **Instruction fetch.**
2. **Instruction dispatch to an instruction queue (also called instruction buffer or **reservation stations**).**
3. The instruction waits in the queue until its input operands are available. The instruction is then allowed to leave the queue before earlier, older instructions.
4. The instruction is issued to the appropriate functional unit and executed by that unit.
5. The results are queued.
6. Only after all older instructions have their results written back to the register file, then this result is written back to the register file. This is called the graduation or retire stage.

CURS 5

Mecanisme pentru sincronizarea procesoarelor

1. **Semafoarele** – sunt cel mai des folosite obiecte pentru sincronizare.

Un **semafor** este o variabila partajata, care poate primi numai valori pozitive. Exista doua operatii de baza, care pot fi efectuate asupra unui semafor:

- DOWN (decrementare);
- UP (incrementare).

Operatia de decrementare este efectuata asupra unui semafor de care un proces care isi doreste continuarea executiei. Daca semaforul atinge valoarea 0, procesul este blocat si asteapta intr-o coada incrementarea semaforului.

Exemplu:

Avem: 20 echipe, 10 terenuri, cos de mingi (semafor).

Daca semaforul are 7 pozitii (7 mingi maxim), modul in care se vor desfasura activitatile este:

- 2 echipe sunt necesare pt a incepe un meci si necesita o minge;
- dupa ce primele 14 echipe vor duce semaforul la valoarea 0 (0 mingi in cos), restul echipelor va trebui sa astepte aparitia unei mingi in cos, pt a putea sa-si desfasoare activitatea

Exista doua modalitati de implementare a lucrului cu semafoare, conform protocoalelor de asteptare:

- BUSY_WAIT
- SLEPP_WAIT

Pentru a decide procesul, se asociaza operatiilor DOWN si UP functiile:

- DOWN_wait
- UP_signal

There are two types of wait protocols: busy-wait and sleep-wait.

In **busy-wait protocol**, process stays in the process context register, which continuously tries for processor availability.

In **sleep-wait protocol**, wait protocol process is removed from the processor and is kept in the wait queue. The hardware complexity of this protocol is more than busy-wait in multiprocessor system; if locks are used for synchronization then busy-wait is used more than sleep-wait

Implementari:

BUSY_WAIT:

```
wait(s)
{
    while(s==0) s--;
```

```
signal(s)
{
    s++;
}
```

Operatia *signal(s)* este atomica, adica se executa fara a fi intrerupta, in vreme ce operatia *wait(s)* este atomica doar daca $s > 0$.

Functia *wait(s)* in cazul in care $s = 0$ nu este atomica, pentru ca ar putea bloca procese care incrementeaza semaforul.

Pentru implementarea SLEEP_WAIT, procesele sunt doar suspendate si nu stau in starea de asteptare ocupata, ca in cazul BUSY_WAIT.

SLEEP_WAIT

wait(s)

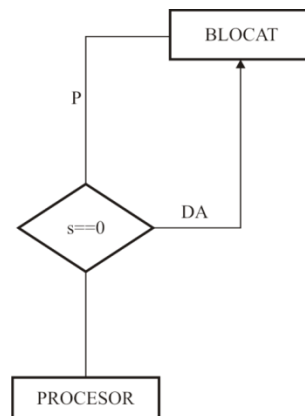
```
{
    if( $s=0$ ) proces suspendat;
    else  $s--$ ;
}
```

signal(s)

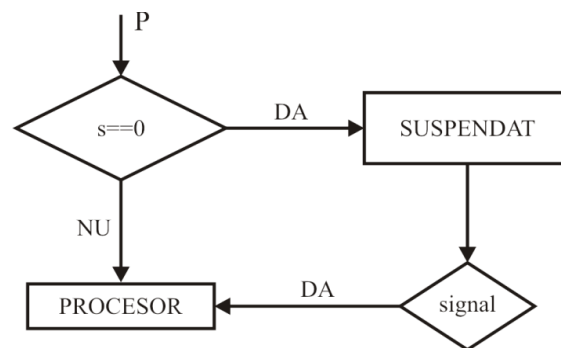
```
{
    if(coada de asteptare nu e goala) planifica proces coada;
    else  $s++$ ;
}
```

In cazul implementarii SLEEP_WAIT, se foloseste o coada de asteptare FIFO, cu procese suspendate. Semaforul este incrementat doar daca coada nu este goala.

BUSY_WAIT



SLEEP_WAIT



2. *Mutex-uri (excludere mutuala)*

Mutex-ul este un caz particular de semafor si controleaza accesul mai multor procese la o resursa partajata.

Exista doua operatii:

- LOCK (accesarea resurse partajate)
- UNLOCK (eliberare resurse partajate)

3. *Evenimentele (semnalele)*

In linux, aparitia unui eveniment s.n semnal. Numarul semnalelor ete de 31 in standardul POSIX.

Un semnal are trei posibile actiuni:

- Tratare
- Ignorare
- Blocare

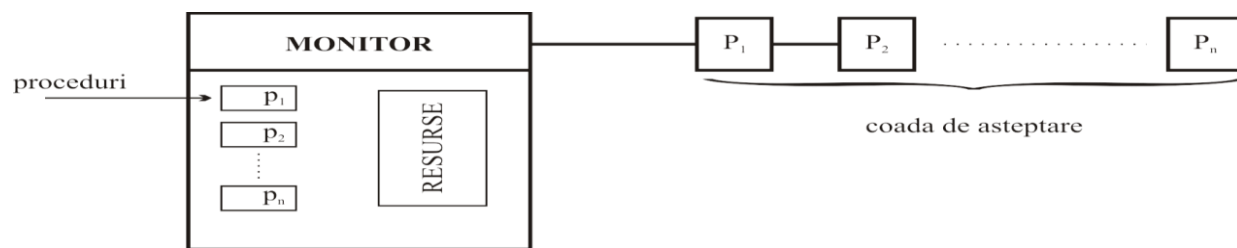
Blocare semnal → implica trecerea acestuia intr-o coada de asteptare.

Ignorare semnal → procesul nu realizeaza nici o actiune pentru ca semnalul este ignorat.

Tratare semnal → executare actiune asociata.

In windows, semnalele s.n evenimente. Acestea pot fi semnalate sau nu.

4. *Monitoare*



Un monitor are 3 componente:

- Numele monitorului si variabilele locale
- Proceduri
- Componenta pentru initializarea variabilelor

Un monitor poate executa un singur proces la un moment dat deci problema excluderii mutuale (mutex) este rezolvata.

Interblocarea

Interblocarea este momentul in care o resursa solicitata de un proces este detinuta de alt proces. Interblocarea dureaza pana la eliberarea resursei.

Resursele pot fi:

- Partajate
- Nepartajate

Resursele partajabile se refera in special la citirea fisierelor.

Resursele nepartajate nu pot fi folosite simultan de de mai multe procese. (exemplu: imprimanta).

Resurse:

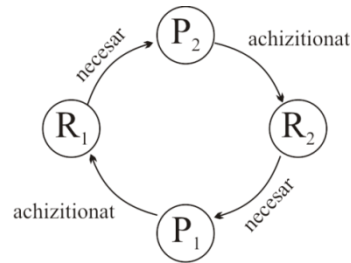
- cu un element
- cu mai multe elemente

Resursele cu mai multe elemente → caz particular, deoarece pot fi accesate simultan de mai multe procese. Ele nu duc intodeana la interblocare.

Pentru utilizarea unei resurse, un proces trebuie sa urmeze etapele:

- cererea de acces la resursa (daca resursa nu este libera, procesul intra in stare de asteptare)

- utilizarea resursei, in care procesul iese din coada de asteptare si utilizeaza datele respective
- eliberarea resursei (resursa devine disponibila)



Problema cinei filosofilor

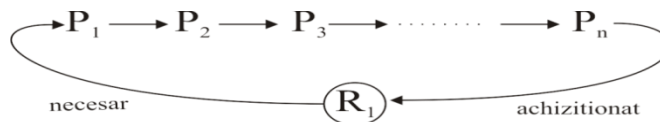
5 filosofi se aseaza la o masa rotunda. Pe masa sunt 5 farfurii si 5 furculite. Un filosof nu poate sa manca decat cu 2 furculite.

Filosoful are 2 stari: mananca, gandeste

Dezvoltati un algoritm prin care filosofii sa reuseasca sa manance la un moment dat iar resursele sa fie utilizate cat mai bine.

OBS: Pt a manca, filosoful ridica furculita din dreapta si apoi furculita din stanga. Daca furculita din stanga este ocupata, el pune jos furculita din dreapta.

- Implementarea achizitionarii fortate este descrisa prin faptul ca un proces nu poate achizitiona o resursa decat dupa ce a fost eliberata.
- Asteptarea circulara: anterior am reprezentat asteptarea pt 2 elemente. Pentru n elemente arata astfel:



Graful de alocare a resurselor

Resursele sunt reprezentate prin patrate si procesele prin cercuri.



Exista doua metode de reprezentare a arcelor grafului:

- arc de cerere $\boxed{R} \xrightarrow{\text{cerere}} \textcircled{P}$
- arc de alocare $\textcircled{P} \xleftarrow{\text{alocare}} \boxed{R}$

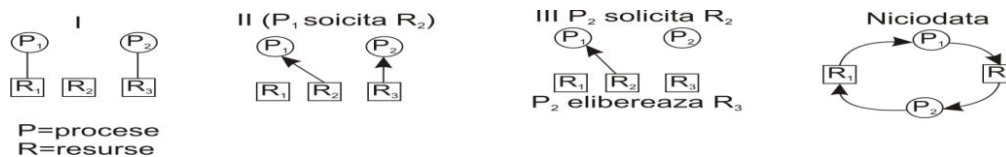
CURS 6

Rezolvarea problemei interblocării

Exista 4 condiții care daca sunt îndeplinite simultan duc la apariția interblocării. Acestea sunt:

- Excluderea mutuala (doua procese nu pot sa acceseze simultan acelasi dispozitiv; de exemplu imprimanta)
- Ocupare si asteptare (un process poate sa mai solicite resurse desi ocupa anumite resurse)
- Imposibilitatea achizitionarii fortate
- Asteptarea circulara

Lipsa unei condiții face ca interblocarea sa poata fi evitata. De exemplu daca cea de a-II-a conditie lipseste interblocarea nu poate sa apara deoarece un proces poate sa solicite maxim o resursa. Simbolic se reprezinta astfel :



In cercuri sunt procese si in dreptunghiuri resurse

Daca a III-a conditie lipseste, procesorul poate sa intrerupa executia unui proces (intrerupere fortata) pentru eliberarea unei sau unor resurse. Interblocarea un poate sa apara daca procesele se executa secvential sau daca nu exista competitie pentru resurse.

Interblocarea poate fi rezolvata prin prin metode:

1. Ignorarea problemei
2. Prevenirea problemei
3. Detectia si recuperarea
4. Evitarea interblocarii

Ignorarea problemei: se bazează pe algoritmul strutului care presupune urmatoarea abordare: ignorarea completa a situatiei in speranta ca lucrurile nu se vor inrautati sau chiar se vor rezolva de la sine. Toate sistemele de operare moderne (Windows,Linux) folosesc aceasta metoda. S-a constatat in practica faptul ca frecventa de aparitie a situatiilor de impas este mult mai mica decât frecventa de aparitie a altor probleme. Un alt motiv este evitarea limitarilor impuse de alte abordari. Pot sa apara probleme la sistemele de operare in timp real.

Prevenirea interblocarii: pentru aceasta metoda este suficient ca una din condițiile de aparitie sa nu fie indeplinita:

- Excluderea mutuala: deoarece multe resurse sunt nonpreemptive (nepartajabile) excluderea mutuala este practic imposibil de evitat. O solutie ar fi implementarea tehnicii **spooling** prin care se poate evita situatia de impas desi se pot atinge limitele sistemelor de stocare a datelor. **Tehnica spooling** este procesul de transferare a datelor intr-o zona temporara de lucru din care

alt proces poate accesa date la un moment ulterior de timp. Tehnica este folosita si in cazul in care se doreste sincronizarea unor echipamente hardware, functionand la viteze diferite.

- Ocupare si asteptare: sunt posibile doua protocoale pentru eliminarea acestei conditii:
 - Un proces un isi incepe executia decat daca are toate resursele necesare.
 - Un proces un poate achizitiona resurse suplimentare decat daca elibereaza resursele curente

Ambele solutii prezinta dezavantaje: Sistemul nu este utilizat la maxim; Apare infometarea proceselor prin care se poate astepta la infinit alocarea unei resurse;

- Imposibilitatea alocarii fortate: poate duce la o executie haotica a proceselor. In practica resursele pot fi preluate doar de la procesele aflate in stare de asteptare.
- Prevenirea asteptarii circulare: presupune introducerea unei ordini in resurse:
 - HDD
 - Imprimanta
 - CD-ROM
 - Scanner

Un proces poate sa utilizeze resursa cu nr de ordine k daca elibereaza toate resursele cu indici mai mari decat k.

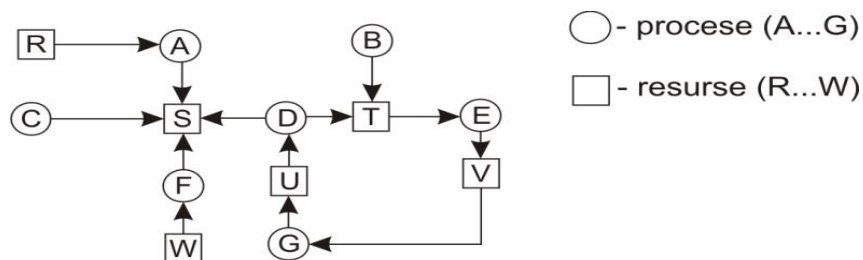
Ex: procesul1 solicita resursa 2 → procesul 1 elibereaza resursele 3 si 4.

Detectia si recuperarea: exista situatii in care algoritmi anteriori un pot fi aplicati. In acest caz se pot executa algoritmi de detectie a interblocarii si de revenire din starea de interblocare. Un sistem in care nu se poate pune in evidente existenta unui ciclu in graful de resurse este un sistem liber de interblocare. Daca exista o singura resursa pentru fiecare tip de resursa se poate aplica următorul algoritim:

Pentru fiecare nod n al grafului se executa pasii urmasori avand nodul n ca nod de start

1. Se initializeaza lista de noduri cu lista vida avand toate arcele nemarcate
2. Se adauga nodul curent in lista; daca apare de 2 ori s-a identificat un deadlock(oprire; STOP)
3. Daca exista arce nemarcate care pleaca din nodul curent se continua cu pasul 4 altfel cu pasul 5
4. Se marcheaza unul din arce ca fiind arcul curent si pornind din acel nod se continua cu pasul 2
5. Elimina nodul din lista si mergi la nodul precedent; daca este nodul initial → Opre; altfel mergi la nodul 3

Ex: $P_1 \leftarrow 2 \Rightarrow P_1$ elibereaza 3,4



Ne propunem parcurgerea grafului din coltul din stanga sus.

Pornire din: $R : L = \{ \} ; L = \{ R, A, S \} ; L = \{ R, A \} ; L = \{ R \}$

$A : L=\{\}, L=\{A,S\} ; L=\{A\}$

$B : L=\{\} ; L=\{B,T,E,V,G,U,D,S\} ; L=\{B,T,E,V,G,U,D,T\}$ STOP (am detectatu un deadlock pentru ca se repeta T)

Algoritmul bancherului

Daca exista mai multe resurse de același tip se folosește un algoritm similar in care resursele sunt grupate in clase de resurse. Vom avea vectori si matrici pentru a putea descrie clasele respective de resurse.

P_i =procesele

E_i =resursele (toate resursele)

A =resursele disponibile

C = matricea de alocare curenta

R =matricea de cereri

$(E_1 \dots E_n)$

$(A_1, A_2, \dots, A_n)$

$$\begin{bmatrix} C_{11} & \dots & C_{1n} \\ \vdots & \ddots & \vdots \\ C_{m1} & \dots & C_{mn} \end{bmatrix}$$

$$\begin{bmatrix} R_{11} & \dots & R_{1n} \\ \vdots & \ddots & \vdots \\ R_{m1} & \dots & R_{mn} \end{bmatrix}$$

Linile din matrici identifica elementele alocate procesului P_i si resursele cerute.

Algoritm:

1. Se cauta un proces nemarcat astfel incat linia sa din matricea de cereri sa aiba toate elementele mai mici sau egale decât elementele vectorului de resurse disponibile.
2. Daca exista un asemenea proces el este marcat si resursele sale din matricea de alocare curenta sunt eliberate (adică adunate la vectorul A). Linia corespunzătoare din matricea de alocare curenta devine 0.
3. Daca nu exista proces atunci STOP.
4. La terminarea algoritmului toate procesele care nu sunt marcate sunt in situatie de impas.

Exemplu:

Consideram vectorul cu toate resursele $E=(4 \ 2 \ 3 \ 1)$ si resursele disponibile $A=(2 \ 1 \ 0 \ 0)$. Matricile de alocare curenta C si cea de cereri R sunt:

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix} \quad R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Prin urmare algoritmul pentru alocarea resurselor este urmatorul:

Pasul 1: procesul P_3 are linia din matricea de cereri R mai mica sau egala cu matricea A deci cererea acestui proces poate fi satisfacuta, deci:

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A=(2 \ 2 \ 2 \ 0)$$

C

Pasul 2: dupa eliberea resurselor de catre P3, procesul P2 poate fi satisfacut, deci C si A devin:

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A=(4 \ 2 \ 2 \ 1)$$

C

In final procesului P1 ii poate fi satisfacuta cererea. Dupa terminarea executiei acestuia matricea C devine 0, toate cererile au fost satisfacuate, iar resursele in final au devenit (4,2,3,1) adica s-au eliberat toate resursele.

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A=(4 \ 2 \ 3 \ 1)$$

C

In concluzie, urmand algoritmul, sistemul nu are situatii de impas daca se executa in ordinea P₃,P₂,P₁.

Exercitiu: Care este situatia in care sistemul ajunge in impas?

Raspuns: Daca procesul P₃ ar solicita (2 0 1 0), desi resursele solicitate sunt mai mici decat vectorul E de resurse totale, sistemul ar fi pus in stare de impas(dead-lock) pentru ca are prea multe deja alocate.

Detectia unui dead-lock nu rezolva insa problema impasului fara mecanisme de recuperare. Recuperarea se executa in momentele de inactivitate a sistemului. O executie frecventa a tehnicilor e recuperare insa ar duce la diminuarea performantei sistemului.

Mecanisme de recuperare (pentru deblocare):

1. **Preemptiunea:** sistemul obtine resurse in mod fortat de la anumite procese pentru a debloca alte procese.
2. **Eliminarea proceselor:** se incearca inchiderea proceselor minimizand consecintele asupra sistemului.
3. **Revenirea prin folosirea punctelor de verificare:** in Windows aceste solutii se pot aplica datele care pot fi restaurate rapid. Exemplu: memorie si registrii. La aparitia starii de impas (dead-lock) procesele care au fost blocate vor fi resturate la un monent anterior de timp pe baza punctelor de verificare. Resursele astfel eliberate vor fi folosite de alte procese.

CURS 7

Rezolvarea problemei interblocării prin Evitarea interblocării

Starea unui sistem înseamnă:

- Cate resurse sunt alocate fiecarui proces
- Care este numarul maxim de resurse pe care un proces le poate cere
- Cate resurse sunt disponibile

:

Am vazut ca prevenirea interblocării foloseste restrictii asupra modului in care se fac cererile de resurse. Evitarea foloseste informatii suplimentare referitoare la cererile de acces. Informatiile necesare pentru evitarea interblocării ajuta la determinarea proceselor care primesc numarul maxim de resurse de care ar avea nevoie un proces pentru a-si termina executia.

Se definesc doua notiuni:

Stare sigura. O stare este sigura daca nu exista un dead-lock si procesele pot fi planificate intr-o ordine astfel incat sa isi poata termina activitatea.

Secventa sigura

$P_1 \dots P_n$ in aceasta ordine se presupune ca secventa este sigura, daca pentru orice P_i $1 \leq i \leq n$, daca ar solicita numarul maxim de resurse si numarul de resurse detinut de proces este mai mica decat numarul de resurse disponibile.

In situatia in care toate resursele sunt de un singur tip, numarul de resurse disponibile se calculeaza prin insumarea resurselor alocate proceselor $1 \dots P_{i-1}$



Disponibile: 6 resurse - 4 resurse alocate = 2 resurse disponibile

P_1 (are 2) si solicita 1	(P_1)	
P_2 (are 1) si solicita 3	←	Disponibile $2+2=4$ resurse
	(P_1)(P_2)	
P_3 (are 1) si solicita 5	←	Disponibile $2+2+1=5$ resurse

Concluzie : In acest exemplu secventa sigura este P_1, P_2, P_3 .

Intrebare: Este secventa P_2, P_1, P_3 sigura? **Raspuns:** NU.

Este posibil ca o stare sigura sa conduca la o stare nesigura in functie de ordinea de planificare a proceselor.

	P_1	P_2	P_3	P_4	P_5
MAX:	10	15	20	25	30
INITIAL:	5	5	10	10	20
TOTAL:	60				

Calculati daca secventa $P_1 P_2 P_3 P_4 P_5$ este sigura.

Resurse disponibile = $10(60 - 5 - 5 - 10 - 10 - 20)$

$P_1(5)$ solicita 5, disp 10
 $P_2(5)$ solicita 10, disp 15
 $P_3(10)$ solicita 10, disp 20
 $P_4(10)$ solicita 15, disp 30
 $P_5(20)$ solicita 10, disp 40
 Este sigura.

Intrebare Este secventa P_4, P_1, P_2, P_3, P_5 sigura? NU.

Disp 10, $P_4(10) \leftarrow 15 \Rightarrow \times$ (dead-lock) ($P_4(10) \leftarrow 15 = P_4(10)$ are 10 si solicita 15)

Algoritmul bancherului

A fost propus de Dijkstra. Este utilizat pentru satisfacerea cererilor unui proces. Daca se atinge o stare nesigura cererea procesului este refuzata. Pentru a verifica daca o stare este sigura, bancherul verifica in prealabil daca resursele disponibile sunt suficiente pentru cererile unui client. Fiecare stare este actualizata imediat dupa aparitia ei. Cererile care duc la impas sunt refuzate sau amânate.

Algoritmul bancherului pentru o resursa

Pentru acest algoritm:

- o cerere care conduce la o stare nesigura este refuzata/amanata pana cand apar resurse.
- dintr-o stare sigura bancherul trebuie sa verifice daca resursele ramase pot satisface cel putin un client.

Exemplu 1: se considera numarul total de resurse 10. Avem 3 procese (A,B,C) care au urmatoarele resurse alocate si urmatorul necesar maxim de resurse $M=[9,4,7]$, fiind disponibile $10-3-3-2=2$ res.:

Are Max			pas 1			pas 2			pas 3 se termina
A	3	9	A	3	9	A	3	9	
B	3	4	B	-	-	B	-	-	
C	2	7	C	2	7	C	7	7	
Disp.2			Disp.1			Disp.10			
						Disp.7			

Obs.: Daca procesul A mai primeste o resursa atunci se va atinge o stare nesigura, deci aplicand algoritmul bancherului oricare cerere de la A va fi refuzata. In mod similar si C va fi refuzat, chiar daca alocarea unei singure resurse nu afecteaza pe moment starea sistemului.

Exemplu 2: consideram numarul maxim tot de 10 resurse si alocarea A4 B3 C2 cu necesarul A9B4C7

Are Max								
A	4	9	A	4	9	A	4	9
B	3	4	B	4	4	B	-	-
C	2	7	C	2	7	C	2	7
Resurse sp.1			Disp.0			Disp.4		

Pentru exemplul 2 nu poate fi gasita nici o secventa sigura; deci trebuie sa gasim alte mecanisme (de exemplu terminarea fortata a unui proces).

Exemplul 3: consideram numarul maxim tot de 10 resurse si necesarul maxim de resurse:

A	0	6
B	0	5
C	0	4
D	0	7
Disp. 10		

Pentru urmatoarea cerere de alocare a resurselor (A-1; B-1; C-2; D-4), este starea rezultata sigura?
Care process poate continua?

Raspuns: C,B,A,D
C,B,D,A

C,D,B,A

Problema: Dacă inițial avem (1,2,2,4) verificați dacă avem secvență sigură.

Răspuns: dead-lock

	Initial	Maxim	Solicita
A	45	70	25
B	40	60	20
C	15	60	45
Disp. 150			

Presupunând că apare procesul D care solicită 25 resurse și are nevoie de maxim 60 resurse. Este starea rezultată sigură?

R: Starea este sigură ($150 - 25 = 125$)

Întrebare: Care este numărul maxim de resurse inițiale pentru D (având același necesar maxim de resurse), pentru ca sistemul să fie încă sigur?

Obs. Așa cum s-a prezentat:

P_i = procesele

E_i = resursele (toate resursele)

A = resursele disponibile

C = matricea de alocare curentă

R = matricea de cereri

Algoritmul bancherului pentru resurse multiple

Vom folosi următoarele notații generalizate :

C = matricea curentă de alocare

R = matricea de cereri

A = vector resurse disponibile

E = vector resurse existente

P = vectorul proceselor

Algoritmul :

- Căutăm o linie în matricea de cereri pentru care resursele necesare sunt mai mici sau egale cu valorile din vectorul A de resurse disponibile. Dacă nu există se poate intra în starea de impas pentru că nici un proces nu își termină activitatea.
- Dacă procesul din linia selectată cere toate resursele și apoi își termină activitatea, acestea vor fi adăugate la vectorul A și procesul este marcat drept terminat/
- Se repetă pașii a) și b) până când toate procesele sunt marcate drept terminate (conducând la starea inițială sigură) sau nu mai există procese ale căror cereri nu mai pot fi satisfăcute (dead-lock).

Exemplu 1 : Se consideră trei procese cu matricea de cereri R , matricea de alocare curentă C și A vectorul de resurse disponibile.

$$\begin{matrix} & R \\ \begin{matrix} A \\ B \\ C \end{matrix} & \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 2 & 1 & 0 & 0 \end{bmatrix} \end{matrix} \text{ matricea de cereri}$$

$$\begin{matrix} & C \text{ alocate} \\ \begin{matrix} A \\ B \\ C \end{matrix} & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix} \text{ alocări curente}$$

Vectorul de resurse disponibile $A=[2, 1, 0, 0]$.

Se va determina E si daca este situatie de impas sau exista o stare sigura.

Observatie: daca pornim cu procesul C se ajunge la o stare sigura

Exemplu: Se considera 5 procese cu matricea de cereri R, matricea de alocare curenta C si vectorul de resurse disponibile A.

$$\begin{array}{c}
 \begin{array}{c} \text{C} \\ \begin{matrix} P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \end{matrix} \end{array} \begin{bmatrix} 0 & 0 & 1 & 2 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 3 & 4 \\ 2 & 3 & 5 & 4 \\ 0 & 3 & 3 & 2 \end{bmatrix} \\
 \text{alocari curente}
 \end{array}
 \quad
 A=[2, 1, 0, 0]=\text{resurse disponibile}
 \quad
 \begin{array}{c} \text{R} \\ \begin{bmatrix} 0 & 0 & 1 & 2 \\ 2 & 7 & 5 & 0 \\ 6 & 6 & 5 & 6 \\ 4 & 3 & 5 & 6 \\ 0 & 6 & 5 & 2 \end{bmatrix} \end{array}
 \Rightarrow E? \text{ secventa sigura?}$$

$$E=[6, 7, 12, 12]$$

$$\begin{bmatrix} 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 4 \\ 2 & 3 & 5 & 4 \\ 0 & 3 & 3 & 2 \end{bmatrix}$$

Secventa: P_1

$$A(P_1) = [2 \ 1 \ 1 \ 2] = A + C(P_1)$$

$$A(P_1, P_4) = [4 \ 4 \ 6 \ 6] = A + C(P_4)$$

$$A(P_1, P_4, P_5) = [4 \ 7 \ 9 \ 8] =$$

$$A(P_1, P_4, P_5, P_2) = [6 \ 7 \ 9 \ 8] =$$

$$A(P_1, P_4, P_5, P_2, P_3) = [6 \ 7 \ 12 \ 12]$$

↓
secventa sigura

max - C

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 7 & 5 & 0 \\ 6 & 6 & 2 & 2 \\ 2 & 0 & 0 & 2 \\ 0 & 3 & 2 & 0 \end{bmatrix}$$

Exemplu: $E = [6 \ 3 \ 4 \ 2]$ (total resurse)

$$MAX = \begin{bmatrix} 4 & 1 & 1 & 1 \\ 0 & 2 & 1 & 2 \\ 4 & 2 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 2 & 1 & 1 & 0 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (\text{alocare curenta})$$

Secventa sigura

$$A = [1 \ 0 \ 2 \ 0]$$

$$R = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 2 \\ 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 2 & 1 & 1 & 0 \end{bmatrix}$$

$$A < R(i): A(P_4) = [2 \ 1 \ 2 \ 1]$$

$$A(P_4, P_5) = [2 \ 1 \ 2 \ 1]$$

$$A(P_4, P_5, P_1) = [5 \ 1 \ 3 \ 2]$$

$$A(P_4, P_5, P_1, P_2) = [5 \ 2 \ 3 \ 2]$$

$$A(P_4, P_5, P_1, P_2, P_3) = [6 \ 3 \ 4 \ 2]$$

→ secventa sigura P_4, P_5, P_1, P_2, P_3

Dupa P_4 , P_1 sau P_5 pot fi satisfacute. Pentru acest exemplu, desi starea initiala C poate fi satisfacuta acea stare nu conduce la o secventa sigur

CURS 8

COMUNICAREA INTRE PROCESE COOPERANTE

Intre doua procese pot exista doua tipuri de comunicare:

- Comunicarea prin memorie partajata
- Prin sisteme de mesaje

Comunicarea prin memorie partajata

Exista o memorie comuna prin care procesele cooperante pot comunica folosind variabile partajate. Mecanismul este simplu. Un proces scrie intr-o variabila, alt proces o citeste. Raspunderea programarii corecte revine programatorului. Din acest punct de vedere, sistemele de operare monoprocesor si multicalculator nu difera esential.

Prin sisteme de mesaje

Un sistem bazat pe transmiterea de mesaje implementeaza doua operatii: SEND si RECIVE. Sistemul este specific multicalculatoarelor.

Sistemele de operare multicalculator sunt mai complexe si se numesc sisteme de operare distribuite.

Sistemele bazate pe transmiterea de mesaje necesita:

- Stabilirea unui **protocol** pt receptia mesajelor ~ un protocol este necesar deoarece in sistemele distribuite mesaje pot fi pierdute. Daca nu se primeste confirmarea receptiei unui mesaj, acesta trebuie retransmis.
- Evitarea receptionarii mesajelor multiple
- Numirea proceselor: este necesara pentru a evita orice ambiguitate.

Aceasta cerinta este legata de procesul de autentificare

Sistemele distribuite introduc notiunea de "**linie de comunicatie**". Problemele implementarii liniei de comunicatie sunt :

- Stabilirea liniei de comunicatie;
- Nr de procese asociat unei linii
- Capacitatea liniei
- Directionalitatea liniei (uni/bidirectionala)

Din punct de vedere logic proiectarea liniei de comunicatie se poate realiza prin **buffering implicit** sau **explicit, prin** comunicatie simetrica sau asimetrica, prin comunicatie directa sau indirecta.

Comunicatia directa este cea in care procesele care trimit sau receptioneaza mesaje trebuie sa mentioneze numele procesului.

- SEND (proces,mesaje)
- RECIVE (proces, mesaj)

Comunicatia directa presupune ca exista un nr fix de mesaje in sistem, procesul care trimite mesajul se numeste **producator**, procesul care primeste mesajul se numeste **consumator**.

O implementare asimetrica a comunicatiei directe este:

- SEND (proces, mesaj)
- RECIVE (identificator, mesaj)

Comunicarea indirecta implica folosirea unei casute de mesaje. Primitivele se modifica:

- SEND (port, mesaj)
 - RECEIVE (port, mesaj)
- Casuta postala are la un moment dat un singur proprietar, desi se cunoaste implicit numele procesului.

Linia de comunicatie are drept caracteristica principala **capacitatea**.

- **Capacitatea 0** : nu exista buffer, desi procesul emitator va fi blocat pana la primirea mesajului. Acest proces de comunicare se mai numeste si rendez-vous.
- **Capacitatea limitata** : exista un buffer cu n elemente (este cea mai intalnita).
- **Capacitatea ∞** : in acest caz **procesul** nu va trebui sa astepte.

Implementarea comunicatiei client-server de la laborator:

- send (S, mesaj)
- receive (S, mesaj) //confirmare receptie
- send (C, mesaj)
- receive (C, mesaj) // ACK confirmare

Capacitatea liniei este 0 deoarece procesul emitator se blocheaza pana la receptia confirmarii

Exceptii in comunicare

In sistemele cu memorie partajata, aparitia unei erori duce la blocarea sistemului.

In sistemele distribuite, erorile nu sunt grave dar trebuiesc tratate.

Exista trei situatii (exceptii):

- Terminarea unui proces inainte de primirea mesajelor ~ in acest caz, pot apare situatii in care mesajele nu ajung la destinatie sau procesele asteapta mesaje care nu vor fi primite. In asemenea situatii, fie SO termina fortat procesul. SO comunica faptul ca procesul s-a incheiat. Problemele pot sa apara cand capacitatea liniei este 0, iar procesul corespunzator (comunicant) este blocat.
- Pierderea mesajelor este datorata defectiunii liniei de comunicatie si poate avea 3 rezolvari:
 - Depistarea si retransmisia mesajelor spre SO.
 - Depistarea si retransmiterea de catre procesul emitent.
 - Detectarea de catre SO si decizia procesului emitent.
- Amestecarea mesajelor ~ mesajele transmise sunt modificate la receptie in mod eronat, desi nu vor putea fi citite corect. Solutia este folosind codul lor detectoare de erori (de ex: bitul de paritate, suma de control, suma de verificare CRC).

Probleme clasice de coordonare si sincronizare a proceselor:

- Problema producator-consumator;
- Problema barbierului somnoros;
- Problema filosofilor chinezi;
- Problema cititori-scriitori
- Problema gradinilor ornamentale.

Problema producator-consumator

Fie o serie de procese concurente care produc date (producator) consumate de procesele consumator. Datele sunt consumate in ordine iar viteza de productie difera de viteza de cosum. Solutia este simpla daca bufferul ar fi infinit.

Exista 2 solutii:

- prin folosirea de semafoare
- prin folosire mesaj.

```
void producer()
{
    int item;
    while(1)
    {
        item = create() //obiectul este creat;
        if(count==N) sleep(); // producatorul se opreste;
                                // doar cand a fost atinsa dimensiunea maxima a buffer-ului
        add_item(item); // operatia se executa doar daca producatorul a fost trezit
        count ++;
        if(count == 1) wakeup(consumator);
    }
}
```

```
// consumatorul
void consumator()
{
    int item;
    while(1)
    {
        if(count == 0) sleep();
        item=getitem();
        count --;
        if(count == N-1) wakeup(producer);
        use_item(item);
    }
}
```

Producatorul este blocat cand buffer-ul este plin, iar consumatorul cand buffer-ul este gol. Deblocarea are loc atunci cand conditia nu mai este de actualitate.

Probleme:

- Accesarea buffer-ului se face secvential;
- Nu exista constrangeri asupra variabilei count;
- Prin folosirea operatiilor intretesute, exista posibilitatea pierderii unui apel de trezit, iar amandoua procesele vor ajunge in scurt timp blocate.

Solutia consta in folosirea semafoarelor prin controlul accesului la variabila count sau prin mesaje.

CURS 9

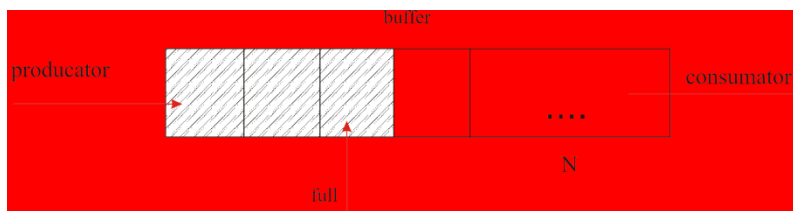
Implementare problemei producator consumator cu semafoare si mesaje

Rezolvarea problemei producator-consumator prin semafoare

Semafoarele au fost introduse de Dijkstra si au rolul de a evita pierderea semnalelor de blocare (de trezire). Semafoarele sunt folosite si pentru a numara cererile de trezire realizate. Semaforul este un numar intreg non-negativ asupra caruia se pot efectua 2 operatii (down() si up()). Daca se incearca decrementarea unui semafor aflat in 0, procesul este blocat.

Problema producator-consumator poate fi modelata cu semafoare astfel:

- un semafor FULL: numarapozitiile umplute (initial full=0)
- un semafor EMPTY: numarapozitiile libere (initial empty=N)



Folosind doar aceste date problema poate ajunge la impas. Pentru a evita impasul se foloseste o valoare suplimentara binara numita semafor sau **mutex** care controleaza accesul la regiunea critica. Aceasta asigura faptul ca producatorul si consumatorul nu vor accesa bufferul simultan.

```

mutex=1;full=0;empty=N;
voidprodicator()
{
int item;
while(1) {
    item=create();
    down(&empty);
    down(&mutex);
    add_item(item);
    up(&mutex);
    up(&full);
}
}

```

deblocare

Semaforul **empty** este decrementat pentru a indica adaugarea unui element in buffer, iar semaforul **mutex** este decrementat pentru a accesa intrarea in regiunea critica.

```

void consumator()
{
int item;
while(1) {
    down(&full);
    down(&mutex);
    item=get_item();
    up(&mutex);
    up(&empty);
    consuma_item();
}
}

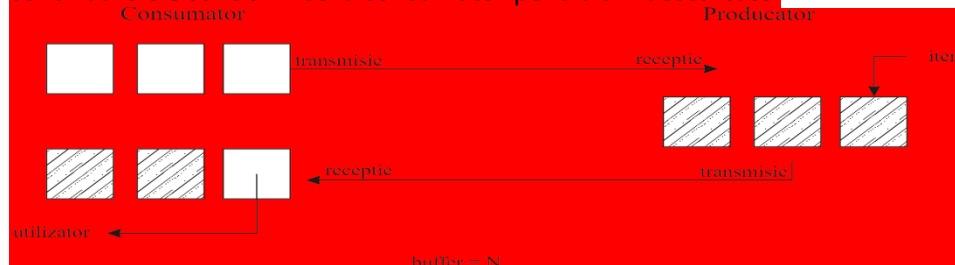
```

bufer full=N =>empty = c => deblocare daca se consuma un item, producator blocat

Daca din greseala ordinea decrementarii era (in producator) inversata prin includerea in regiunea critica semaforul empty, in programul consumator nu se putea intra in regiunea critica, deci nu se putea ajunge la instructiuneaup(&empty).

Implementarea problemei producator-consmator cu mesaje

Solutia se bazeaza pe faptul ca **numarul mesajelor din sistem este fix**. Se considera linia de transmisie de capacitate limitata la N pozitii. Initial mesajele goale sunt transmise de consumator pentru a fi umplute. In continuare ele sunt trimise la consumator pentru a fi descarcate.



```

voidprodicator()

```

```

{
int item;
messagemsg;
while(1)
{
    item=create_item();
    receive(consumer,msg);
    generate(&msg,item);
    send(consumator,&msg);
}
}

void consumator()
{
int item;
messagemsg;
for(i=0;i<N;i++)
send(producator, &msg);
while(1)
{
    receive(producator,&msg)
    item=get_item();
    send(producator, &msg);
    use(item)
}
}

```

Probleme tipice pentru comunicarea intre procese

Problema barbierului somnoros

Pravalia unui barbier este formata din 2 camere. Una folosita ca si camera de asteptare, iar alta in care se asezacientii care vor fi serviti. Daca nu are clientibarbierul se culca.Daca sosesc mai multiclientidecatnumarul de scaune in sala de asteptare, acestiupleaca. Sa se simuleze intr-un program activitatea barbierului.

REZOLVARE: Aceasta problema este tipica tuturor sistemelorhelpdesk.Problema este o reformulare a celei anterioare (producator-consumator). Sala de asteptare este bufferul cu N elemente, iar cele 2 conditii sunt:

- vine primul client; trezestebarbierul
- bufferul este plin (scaunele sunt ocupate), noii clientiparasescpravalia

Implementare

Exista 3 semafoare:

- clienti (clientii care vor sa ajunga pe scaunul barbierului)
- barbierul (are 2 valori 0 sau 1 indicandnumarul de frizeri disponibili)
- mutexul folosit in procesul de excludere mutuala

Presupunand ca avem variabila waiting care numara clientii din sala de asteptare avem urmatoarea implementare:

//ce este cu * reprezinta regiunea critica

```
void barbier()
{
while(1)
{
down(&clienti);
* down(&mutex);
* waiting--;
* up(&barbier); //barbier are 0 sau 1
* up(mutex);
servire_client();
}
}
```

//N=nr maxim de scaune din sala de asteptare

//ce este cu * reprezinta regiunea critica

```
void client()
{
* down(&mutex);
* if (waiting<N)
* {
* waiting++;
* up(&client);
* up(&mutex);
down(&barbier);
obține_serviciu();
}
else up(&mutex);
}
```

Problema cititori-scriitori

Problema prezinta modul de accesare al unui obiect partajat de unul sau mai multe procese concurente. Cititorii sunt cele care citesc si scriitorii sunt cei care introduc date in buffer. Cititorii trebuie sa permita accesul simultan, dar scriitorii trebuie sa aiba acces exclusiv.

Exista 2 solutii:

- Nici un cititor nu va fi ținut în așteptare cu excepția cazului în care un scriitor a obținut deja permisiunea de scriere. La un acces simultan cititorii au prioritate. Problema care poate să apară este infometarea a doua așteptarea la infinit a dreptului de acces.
- Scriitorii au prioritate în fața cititorilor.

Problema cinei filosofilor chinezi

Problema se poate generaliza pentru N elemente cu $N > 3$.

Cinci filosofi mănâncă în jurul unei mese circulare. Există 5 furculițe, 5 BETISOARE și un platou cu mâncare central. Un filosof poate mânca decât cu 2 betisoare. Un filosof poate ridica doar o furculiță la un moment dat.

Daca un filosof nu manancagandeste. In final problema trebuie sa rezolve situatiainterblocarii (trebuie evitata asteptarea circulara); trebuie evitata problema infometarii (trebuie la un moment dat sa le dam sa manance).

Problema rezervarii biletelor

Fiecare terminal al unei retele de calculatoare areintr-un punct de vanzare al biletelor. Simulativanzarea biletelor fara a vinde 2 sau mai multe bilete pe acelasi loc.

Problema gradinii ornamentale

Accesul la o gradina ornamentala se face prin N intrari. Tineti evidenta persoanelor care au intrat in gradina. Fiecare poarta este o resursa care trebuie accesata exclusiv, adica la un moment dat prin oricare poarta intra o singura persoana.

Problema emitator-receptor

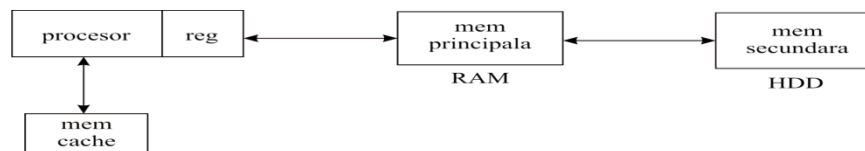
Un emitator trimite succesiv mesaje, fiecare trebuind sa fie primit de toti receptorii. Doar dupa ce procesul s-a terminat emitatorul poate trimite urmatorul mesaj. Aceasta este o problema tipica client-server pentru retele de calculatoare.

CURS 10

Gestionarea memoriei

Sistemele de operare moderne trebuie sa aloce memoria mai multor programe executate in paralel (simultan). El trebuie sa asigure mecanisme pentru protectia memoriei astfel incat programele sa poată coexista in aceasta. Componentele hardware moderne au incorporat multe din mecanismele sistemelor de operare.

Ierarhii de memorie



Memoriile pot fi clasificate in memorii interne (registrii, cache si RAM) si memorii externe (hard-disk-ul). Principalele caracteristici ale memoriei sunt capacitatea si timpul de acces. O alta clasificare in functie de timpul de acces este:

- Memorii rapide (au un timp de acces foarte mic, sunt realizate de tip static SRAM, au costuri foarte mari; ex memoria cache)
- Memorii lente (implementate dinamic-DRAM, au costuri mai scazute; memoria RAM)
- Memori foarte lente (hdd-uri, au costuri extrem de scazute si capacitate foarte mare)

Ierarhizarea memoriilor are rolul de a pune la dispozitia utilizatorului aparent o memorie rapida de capacitate cat mai mare. Acest lucru se realizeaza prin urmarirea zonelor de memorie care sunt utilizate si eliberarea zonelor care nu sunt utilizate de S.O. Acest lucru este necesar pentru ca memoria rapida are o dimensiune mult mai mica decat memoria necesara executiei.

Optimizari in incarcarea si executia unui program

1. **Incarcarea dinamica** = incarcarea rutinelor in memorie doar cand acestea sunt apelate.
2. **Suprapunerile (overlay)** acest mod de lucru permite executia programelor care solicita mai multa memorie decat memoria fizica. Acest lucru se realiza prin partitionarea programului in module care incapeau in memorie. Aceasta tehnica este depasita in sistemele de operare moderne.
3. **Legarea dinamica prin fisierele de tip .dll (windows) si .so(in linux, unix):** aceasta permite compilarea separata a functiilor unui program si incarcarea acestora doar atunci cand sunt apelate. In programul principal exista un element numit stub care face referire la functia din biblioteca externa.

Alocarea memoriei

In sistemele de operare exista un modul separat numit *alocator de memorie* care contabilizeaza zonele libere si zonele alocate, satisface cererea pentru noi zone si reutilizeaza zonele alocate. O data alocata o zona de memorie unui program, acesta o poate partitiona dupa cum o doreste. Din punct de vedere al limbajelor de programare exista:

- Limbaje care nu pot aloca memorie: memoria este determinata inainte de momentul executiei; toate limbajele de programare moderne permit alocarea memoriei
- Limbaje cu alocare si dealocare explicita: cererea de zone de memorie era realizata explicit

- Limbaje cu colectare de gunoi (garbage collection). Utilizatorul nu are nevoie sa specifice eliberarea unei zone de memorie. O serie de functii care se executa in paralel cu programul realizeaza acest lucru.

Avantaje:

- Se previne aparitia unor erori de programare
- Se previne accesarea unor zone de memorie nealocate

Dezavantaj: Ocuparea memoriei nu este predictibila in timp

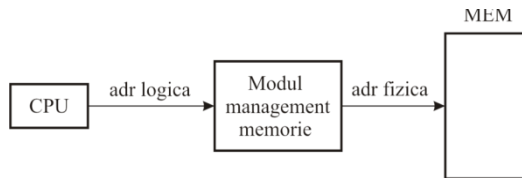
Caracteristici ale alocatoarelor de memorie

1. **Timpul de operatie:** depinde de dimensiunea memoriei deoarece trebuie efectuate mai multe operatii pentru alocarea unei zone de memorie mai mari
2. **Fragmentarea:** se datoreaza faptului ca mici fragmente din memorie raman neutilizate. Se numeste fragmentare interna daca exista spatiu nefolosit in zona de memorie alocata unui program. Se numeste fragmentare externa momentul in care avem zone libere de memorie, dar nici un program nu incapa complet in acestea. Pentru aceasta a doua problema solutia este **compactarea memoriei**
3. **Concurenta:** specifica sistemelor multiprocesor cu memorie partajata. Un alocator bun permite exploatarea resurselor sistemului mult mai bine
4. **Gradul de utilizare:** alocatorul are structuri de date in memorie. Pe masura ce structurile sunt mai mari cu atat eficacitatea scade.

Tipuri de alocare a memoriei:

Exista doua tipuri de adrese:

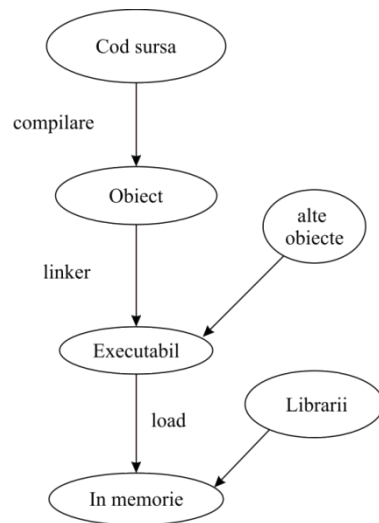
- Adrese fizice (este cea vazuta de modulul de management al memorie)
- Adrese logice (este generata de procesor si mai poarta numele de adresa virtuala)



La momentul compilarii si incarcarii cele doua adrese coincid. Datorita cererilor de alocare de memorie din timpul executiei, in final cele doua ajung sa difere.

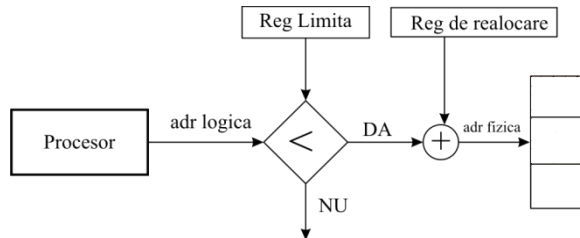
Terminologie:

1. **Re-localizabil (reallocatable):** termenul arata ca zona de memorie ocupata poate fi plasata oriunde in spatiul de memorie.
2. **Binding:** reprezinta procesul prin care programului i se alocă memorie fizica. Poarta denumirea si de mapare adresa logica in adresa fizica.
3. **Compiler:** daca se cunoaste locul in care va fi plasat tot codul programului se genereaza cod absolut; pana la terminarea programului adresele fizice vor coincide cu cele logice; daca este cazul se genereaza cod relocabil;
4. **Load:** Prin aceasta operatie codul relocabil isi gaseste adrese fizice



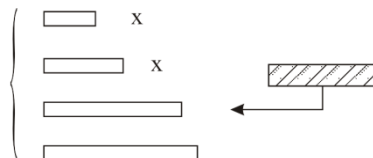
Alocarea memoriei se poate face in mod continuu sau necontinuu. Pentru sistemele cu alocare continua exista urmatoarele scheme de alocare a memoriei:

- **Alocarea unica:** este cea mai veche forma de alocare a memoriei; primele sisteme de operare aveau o singura zona de memorie si anume cea in care se executa programul; sistemele de operare mai noi (MS-DOS) au 2 zone (2 partiti) de memorie: una pentru sistemul de operare si una pentru utilizator.
- **Alocare cu partiti fixe:** se mai numeste si alocare statica in care memoria este impartita in zone nu neaparat egale, iar in fiecare partiie poate rula cel mult un program; programele sunt incarcate in memorie prin cozi de intrare. Dezavantajul acestei scheme este ca raman zone de memorie neutilizate. Schema a fost utilizata de sistemele de calcul Felix dar a fost depășita.

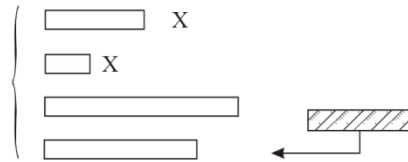


- **Alocare cu partiti variabile:** numărul, locația si dimensionarea partițiilor se face dinamic; din acest motiv utilizarea memoriei este destul de buna. Sistemul de operare trebuie sa mentina o tabela cu partiile libere si una cu cele ocupate. Exista 3 algoritmi pentru alocarea partiilor libere:

➤ First Fit Algorithm (FFA) se parcurge tabela de partiti in sens crescator si se alocă prima potrivire



➤ Best Fit Algorithm (BFA): se parcurge toata lista spatiilor libere si se alege spatiul cu dimensiunea cea mai mica; algoritmul seamana cu FFA daca partiile sunt ordonate crescator; daca nu este un proces de cautare



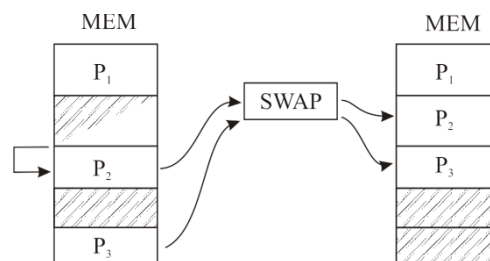
Acest algoritm este folosit de sistemul de operare MS-DOS.

➤ **Worst Fit Algorithm (WFA):** unui proces i se alocă cel mai mare spațiu de memorie disponibil. WFA are gradul cel mai mic de alocare al memoriei, iar First Fit este cel mai rapid.

- **Alocarea prin swapping:** Aceste sisteme de operare au pe unitatea de stocare un spațiu în care descarcă procesele în așteptare din memorie. Ordinea în care sunt evacuate procesele se bazează pe priorități; astfel procesele cu prioritate scăzută sunt primele evacuate. Necesarul de memorie SWAP trebuie să fie mult mai mare decât al memoriei fizice. În practică nu se observă câștiguri semnificative dacă memoria SWAP depășește de 1.5 ori memoria fizică. Dezavantajul acestor sisteme este legat de viteza de descărcare și încărcare a proceselor.

Avantajele sunt multiple:

- a) Pot fi gestionate programe care solicită spații mari de memorie
- b) Se poate realiza compactarea memoriei

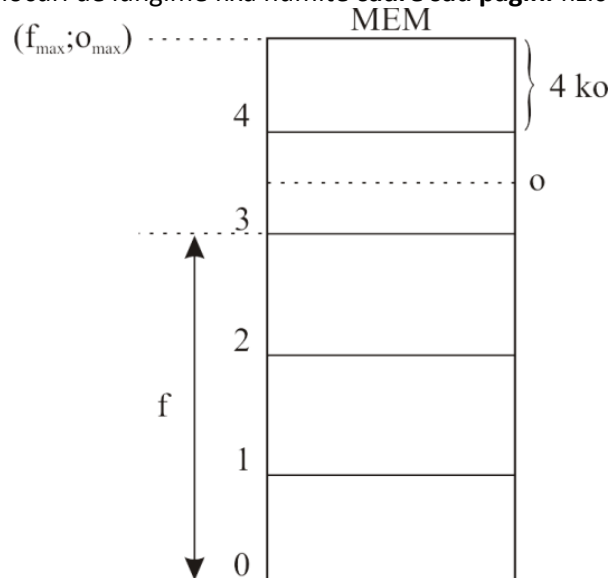


Acest procedeu este dificil de realizat dacă se realizează referiri directe la locațiile de memorie.

CURS 11

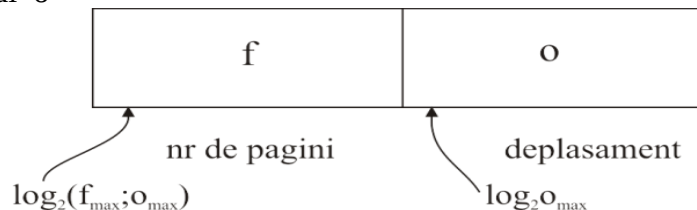
Paginarea memoriei

Paginarea este de tip alocarea memoriei necontinuu. In acest mod, proceselor le poate fi alocata memorie atat interna (registrii + ram) cat si externa (HDD), dar si faptul ca parti ale unui proces se pot reagasii in diferite segmente de memorie. Suportul hardware pentru aceasta implementare este asigurat pe de o parte de memoria fizica impartita in blocuri de lungime fixa numite **cadre sau pagini** fizice.



Pentru un procesor intel paginarea se face in 4 kb. Lungimea unei pagini trebuie sa fie o putere a lui 2 si este o constanta pentru fiecare arhitectura. Pentru accesarea unei memorii fizice paratjate in acest mod adresa este impartita in 2 componente:

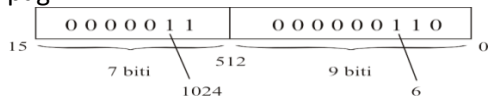
- Nr de pagina f
- Si deplasamentul o



Exemplu: Se considera un sistem cu spatiu de adresare pe 16 biti cu 512 octeti per pagina. Calculati locatia de memorie indicata de adresa (3,6). 3- nr paginii, 6 pentru deplasament.

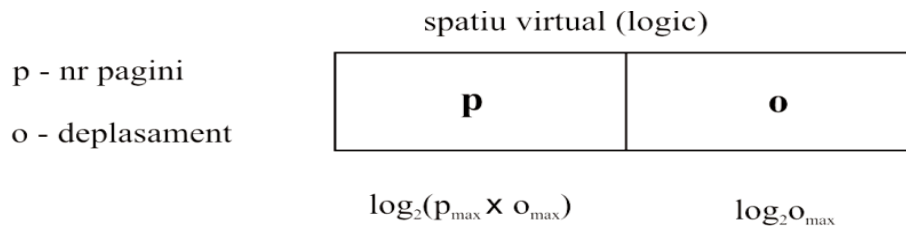
Solutie:

Deoarece sunt 512octeti /pag => $\log_2 512 = 9$ => 9 biti pentru deplasament => $16-9=7$ biti pentru nr de pag.



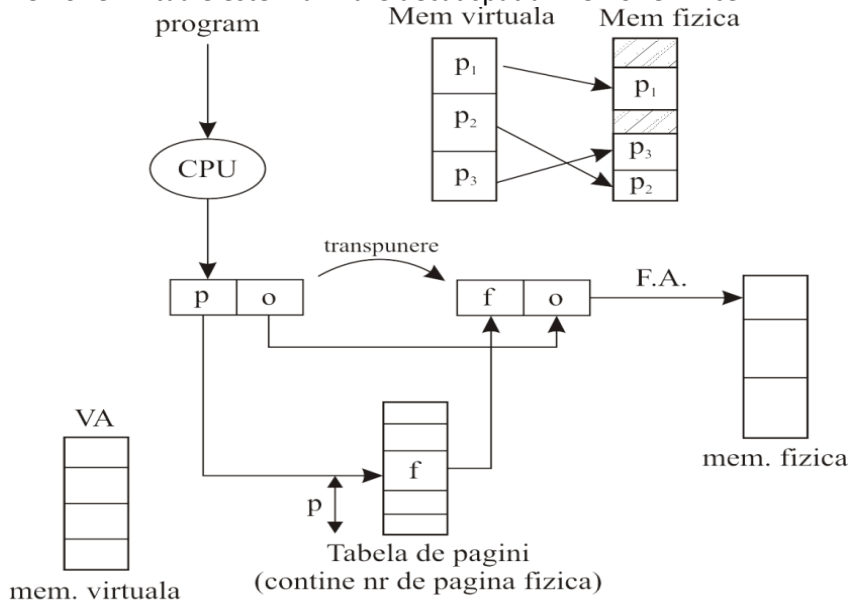
Adresa (3,6) = $3 \cdot 512 + 6 = 1524$

Similar spatiul virtual de adrese este si el paginat.



De obicei, $P_{\max} \approx f_{\max}$

Memoria virtuala este impartita in pagini si avem nr de pag p si deplasamentul in interiorul paginii o.
Spatiul memoriei virtuale este mai mare decat spatiul memoriei fizice.



Datorita faptului ca in spatiul virtual alocarea de memorie este continua, iar in memorie ea este arbitrar incarcata si un toate paginile sunt incarcate in memorie in orice moment, este necesara o tabela de pagini care indica corespondenta intre adresa logica si cea fizica. De obicei este implementata prin registrii cu viteza de acces foarte mare. Practic in desenul nostru, tabela de pagini este in mijloc.

Ex. Care este dimensiunea tabelii de pagini pentru un sistem cu adresare pe 16 biti si cu 512 octeti /pagina?

Solutie:

$512 = 2^9 \Rightarrow 16 - 9 = 7$ biti sunt necesari pentru a accesa pagina

Intrucat la fiecare locatie trebuie plasat numarul paginii (pe 7 biti) $\Rightarrow 2^7 * 7 = 128 * 7 = 896$

Obs

Sistemele moderne de calcul au accesarea memoriei pe 64 sau 128 de biti.

Ex.

Fie un sistem cu adresare pe 16biti, 32ko de memorie fizica si sistem de paginare cu pagina de 1024 octeti. Calculati care este adresa din memoria fizica accesata stiind ca in tabela de pagini in locatia corespunzatoare se afla 000111, iar VA=(3,1023).

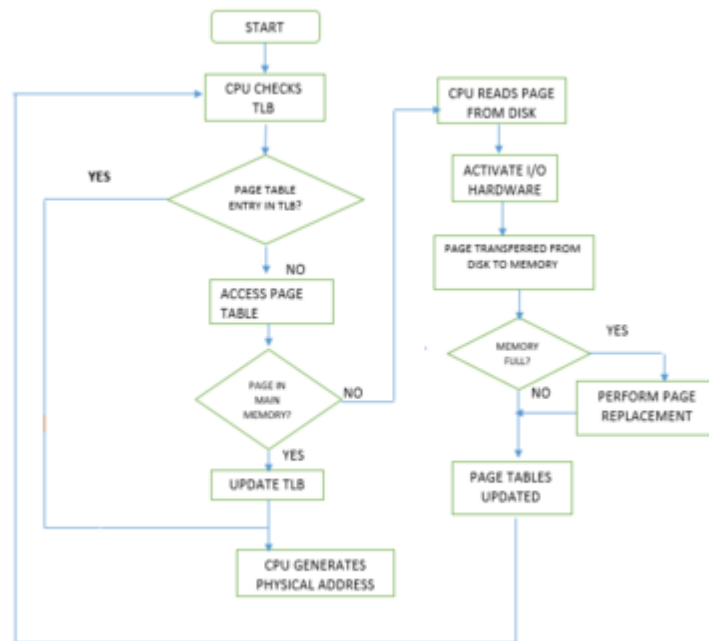
Solutii pentru implementarea tabelii de pagini

- Folosirea registrilor hardware:** este cea mai rapida solutie insa si cea mai costisitoare, deoarece in sistemele moderne tabela de pagini tinde sa ia valori foarte mari.

2. **Folosirea memoriei principale:** costul este mult mai scazut, insa viteza de acces este scazuta; aceasta solutie impune un acces multiplu la memorie, deoarece se acceseaza o data memoria pentru aflarea paginii care trebuie accesata si a doua oara se acceseaza memoria pentru extragerea informatiei dorite. Uneori se foloseste un registru de baza al tabelii de pagini pentru a stoca informatiile despre pagina curenta. Astfel la accesari succesive ale aceleiasi pagini adresa ei se afla intr-un registru (timp de acces foarte mic).
3. **Folosirea unei memorii hardware de tip asociativ:** aceasta memorie foloseste registrii mai mici cu rolul de a stoca adresele fizice folosite recent. Memoria asociativa are 2 componente: o cheie care memoreaza numarul paginii si o valoare indicand cadrul de memorie asociat. Schema de translatare a adreseleor virtuale in adrese fizice devine astfel:

Lucrurile se desfasoara in modul urmatoar:

The flowchart provided explains the working of a TLB. If it is a TLB miss, then the CPU checks the page table for the page table entry. If the present bit is set, then the page is in main memory, and the processor can retrieve the frame number from the page-table entry to form the physical address.^[7] The processor also updates the TLB to include the new page-table entry. Finally, if the present bit is not set, then the desired page is not in the main memory, and a page fault is issued. Then a page-fault interrupt is called, which executes the page-fault handling routine.



A translation lookaside buffer (TLB) is a cache that memory management hardware uses to improve virtual address translation speed.[1] The majority of desktop, laptop, and server processors includes one or more TLBs in the memory management hardware, and it is nearly always present in any hardware that utilizes paged or segmented virtual memory.

The TLB is sometimes implemented as content-addressable memory (CAM). The CAM search key is the virtual address and the search result is a physical address. If the requested address is present in the TLB, the CAM search yields a match quickly and the retrieved physical address can be used to access memory. This is called a TLB hit. If the requested address is not in the TLB, it is a miss, and the translation proceeds by looking up the page table in a process called a page walk. The page walk requires a lot of time when compared to the processor speed, as it involves reading the contents of multiple memory locations and using them to compute the physical address. After the physical address is determined by the page walk, the virtual address to physical address mapping is entered into the TLB. The PowerPC 604, for example, has a two-way set-associative TLB for data loads and stores.[2]

Two schemes for handling TLB misses are commonly found in modern architectures:

- With hardware TLB management, the CPU automatically walks the [page tables](#) (using the [CR3](#) register on [x86](#), for instance) to see whether there is a valid page-table entry for the specified virtual address. If an entry exists, it is brought into the TLB, and the TLB access is retried: this time the access will hit, and the program can proceed normally. If the CPU finds no valid entry for the virtual address in the page tables, it raises a [page fault exception](#), which the [operating system](#) must handle. Handling page faults usually involves bringing the requested data into physical memory, setting up a page table entry to map the faulting virtual address to the correct physical address, and resuming the program. With a hardware-managed TLB, the format of the TLB entries is not visible to software and can change from CPU to CPU without causing loss of compatibility for the programs.
- With software-managed TLBs, a TLB miss generates a *TLB miss* exception, and operating system code is responsible for walking the page tables and performing the translation in software. The operating system then loads the translation into the TLB and restarts the program from the instruction that caused the TLB miss. As with hardware TLB management, if the OS finds no valid translation in the page tables, a page fault has occurred, and the OS must handle it accordingly. [Instruction sets](#) of CPUs that have software-managed TLBs have instructions that allow loading entries into any slot in the TLB. The format of the TLB entry is defined as a part of the instruction set architecture

(ISA).^[11] The [MIPS architecture](#) specifies a software-managed TLB;^[12] the [SPARC V9](#) architecture allows an implementation of SPARC V9 to have no MMU, an MMU with a software-managed TLB, or an MMU with a hardware-managed TLB,^[13] and the UltraSPARC architecture specifies a software-managed TLB.^[14]

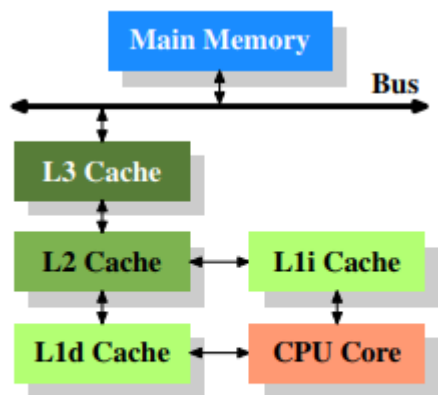


Figure 3.2: Processor with Level 3 Cache

Exercitiu: Calculati diferenta intre timpul de acces la memorie si timpul de acces la un registru din procesor pentru situatia in care memoria are timp de acces 5 nano secunde si frecventa procesorului este de 3GHz. ()
 $3 \cdot 10^9$ vs $5 \cdot 10^9$
 Aproximativ 0.5 nano secunde

Finally, we should at least give an impression of the costs associated with cache hits and misses. These are the numbers Intel lists for a Pentium M:

To Where	Cycles
Register	≤ 1
L1d	~ 3
L2	~ 14
Main Memory	~ 240

in Nehalem, the 64KB of L1 cache has 4 cycle latency, while 256KB of L2 cache (4x more) has 10 cycle latency ($\sim 2\times$ slower).

The biggest changes from the original Sandy Bridge are the increased L3 cache size and the quad-channel memory interface. We'll first look at the impact a 15MB L3 has on latency:

Cache/Memory Latency Comparison

	L1	L2	L3	Main Memory
AMD FX-8150 (3.6GHz)	4	21	65	195
AMD Phenom II X4 975 BE	3	15	59	182

(3.6GHz)				
AMD Phenom II X6 1100T (3.3GHz)	3	14	55	157
Intel Core i5 2500K (3.3GHz)	4	11	25	148
Intel Core i7 3960X (3.3GHz)	4	11	30	167

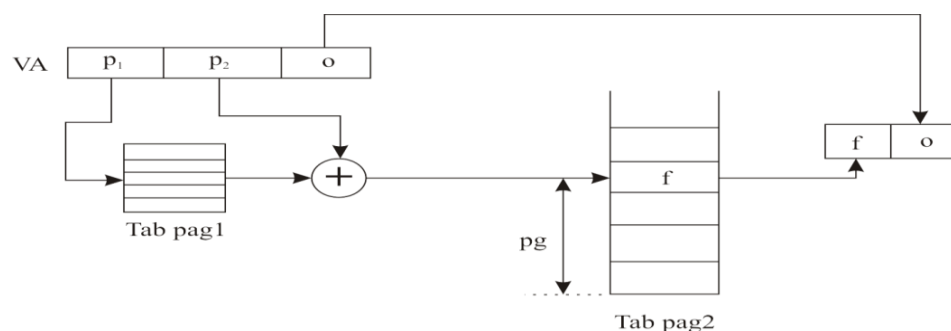
Cachemem shows us a 5 cycle increase in latency. Hits in L3 can take 20% longer to get to the core that requested the data, if this is correct. For small, lightly threaded applications, you may see a slight regression in performance compared to Sandy Bridge. More likely than not however, the ~2 - 2.5x increase in L3 cache size will more than make up for the added latency. Also note that despite the large cache and thanks to its ring bus, Sandy Bridge E's L3 is still lower latency than Gulftown's.

Memory Bandwidth Comparison

	Intel Core i7 3960X (Quad Channel, DDR3-1600)	Intel Core i7 2600K (Dual Channel, DDR3-1600)	Intel Core i7 990X (Triple Channel, DDR3-1333)
Aggregate Memory Bandwidth	37.0 GB/s	21.2 GB/s	19.9 GB/s

Memory bandwidth is also up significantly. Populating all four channels with DDR3-1600 memory, Sandy Bridge E delivered 37GB/s of bandwidth in Sandra's memory bandwidth test. Given the 51GB/s theoretical max of this configuration and a fairly standard 20% overhead, 37GB/s is just about what we want to see here.

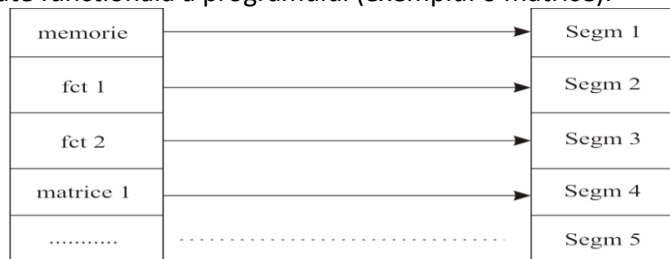
O alta solutie pentru imbunatatirea accesului la tabela de pagini, foloseste accesarea pe niveluri multiple. Astfel memoria virtuala are numarul de pagina impartit in n elemente; n fiind nivelul de cascada. Pentru fiecare nivel exista o tabela de pagini care permite selectia tabelii de pagini din nivelul urmator. In figura este prezentata o implementare a tabelului de pagini pe 2 niveluri.



Impartirea in tabele de dimensiune mai mica face ca operatia de cautare sa se efectueze mai repede.

Segmentarea memoriei

Segmentarea este o schema de administrare a memoriei in care programul este divizat in mai multe parti functionale spatiul logic de adresare al programului fiind astfel segmentat fiecarui segment corespunzandu-i o unitate functionala a programului (exemplu: o matrice).



Fiecare segment are un nume si un deplasament. Prin urmare programatorul vede spatiul virtual de adresare drept un spatiu bidimensional. Aceasta tehnica (segmentarea) permite implementarea unei facilitati numita memorie virtuala. Prin aceasta se alocă executiei programului un spatiu de memorie mai mare decat memoria disponibila. Pentru solutionare se foloseste paginarea combinata cu folosirea memoriei swap si segmentarea.

Problemele care necesita clarificare in acest scop sunt urmatoarele:

1. **Strategia de plasare in memorie** (daca plasarea se face oriunde in memorie nu este nevoie de strategie)
2. **Strategia de inlocuire** (priveste modul de lucru in cazul in care sunt mai multe sarcini decat incap in memorie)
3. **Controlul incarcarii memoriei** (priveste numarul maxim de sarcini care pot fi incarcate in memorie). Cand memoria se umple procesul trebuie trimis in memoria swap. O eroare de pagina (page fault) cauzata din acest motiv conduce la 2 accesari de disc.

In scopul minimizarii accesarii discului trebuie avute in vedere strategii locale de inlocuire a procesorului (descarcarea sau incarcarea in memorie a functiilor unui proces) sau strategii globale (scoaterea din memorie a unor procese in detrimentul altora).

Cele 2 principii prin care se incearca identificarea evolutiei urmatoare sunt localitatea temporală (locatile accesate recent au tendinta de a fi accesate si in viitor) si localitatea spatiala (daca o locatie e accesata sansele sunt mari sa fie accesate si locatii adiacente). Pentru a realiza aceasta predictibilitate se utilizeaza un istoric al accesarii paginilor.

Din pacate nu este posibila predictia precisa a executiei proceselor in timp. Cei mai folositi algoritmi sunt algoritmi FIFO, algoritmul least recently used (LRU) si algoritmul Least Frequently Used (LFU).

Algoritmul FIFO este cel mai simplu de folosit deoarece necesita o lista simpla pentru pastrarea ordinii. Algoritmul are probleme in situatiile in care o pagina frecvent utilizata este evacuata pe disc. Practic daca o pagina se regaseste in lista de pagini de la momentul de timp anterior, se considera HIT si nu se fac inlocuiri; daca pagina nu este in lista anterioara, o va inlocui pe cea mai veche pagina introdusa in tabela.

Ex 1:

	pagini solicitate									
pagini initiale	A	A	A	B	E	F	F	F	C	A
A	A	A	A	A	E	E	E	E	E	E
B	B	B	B	B	B	F	F	F	F	F
C	C	C	C	C	C	C	C	C	C	A
D	D	D	D	D	D	D	D	D	D	D
	H	H	H	H	M	M	H	H	H	M

Ex 2:

	pagini solicitate									
initial in TLB	A	C	D	E	A	A	B	B	E	F
A	A	A	A	E	E	E	E	E	E	E
B	B	B	B	B	A	A	A	A	A	A
C	C	C	C	C	C	C	B	B	B	B
D	D	D	D	D	D	D	D	D	D	F
	H	H	H	M	M	H	M	H	H	M

Algoritmul LRU (Least recently used)

Spre deosebire de FIFO, algoritmul LRU verifica care este cererea care nu a fost accesata de cel mult timp si o inlocuieste daca apare o cerere de pagina noua care nu este in lista. Pentru a rezolva aceasta problema se folosesc liste multiple (pentru a pastra ordinea de accesare).

Ex:

	pagini solicitate									
pagini initiali	A	A	A	B	E	F	F	F	C	A
A	B	B	B	C	D	A	A	A	B	E
B	C	C	C	D	A	B	B	B	E	F
C	D	D	D	A	B	E	E	E	F	C
D	A	A	A	B	E	F	F	F	C	A
	H	H	H	H	M	M	H	H	M	M

Ex:

	pagini solicitate									
initial in TLB	A	C	D	E	A	A	B	B	E	F
A	B	B	B	A	C	C	D	D	D	A
B	C	D	A	C	D	D	E	E	A	B
C	D	A	C	D	E	E	A	A	B	E
D	A	C	D	E	A	A	B	B	E	F
	H	H	H	M	H	H	M	H	H	M

Algoritmul LFU (Least Frequently Used)

Pentru acest algoritm se ia in calcul numarul de accesari al paginii. Paginile candidat pentru inlocuire sunt cele care au cele mai putine accesari. Pentru situatia de egalitate, se aplica regula FIFO, adica pagina cea mai veche este inlocuita. In acst mod paginile frecvent accesate sunt pastrate iar cele rar accesate sunt inlocuite primele.

		pagini solicitate									
	initial in TLB	A	C	D	E	A	A	B	B	E	F
	A	A	A	A	A	A	A	A	A	A	A
	B	B	B	B	E	E	E	E	E	E	E
	C	C	C	C	C	C	C	B	B	B	B
	D	D	D	D	D	D	D	D	D	D	F
		H	H	H	M	H	H	M	H	H	M
frecv acce	0	1	1	1	1	2	3	3	3	3	3
	0	0	0	0	1	1	1	1	1	2	2
	0	0	1	1	1	1	1	1	2	2	2
	0	0	0	1	1	1	1	1	1	1	1

CURS 12

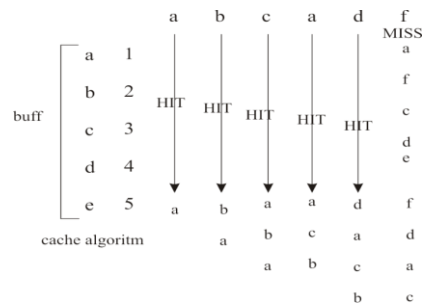
Algoritmi de inlocuire a paginii

<Continuare>

Algoritmul Last Recently Used:

Observatie: Algoritmul LRU inlocuieste in tabelul cache folosit de algoritm paginile daca acestea exista in cache la momentul efectuarii unui hit.

Exemplu:



Algoritmul Least Frequently Used (LFU):

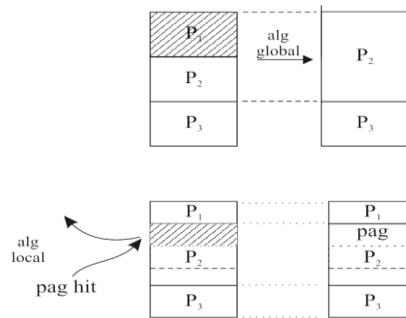
Frecventa accesarii paginilor este contorizata. Contorul nu este resetat, iar candidatul pentru inlocuire este pagina cu contorul cel mai mic. Algoritmul are probleme atunci cand unele pagini sunt des utilizate in stagiile initiale de incarcare ale sistemului de operare.

Algoritmul Real Paged Daemon: De obicei sistemele de operare desemneaza un proces autonom care sta in fundal si este executat periodic pentru implementarea politicii de inlocuire a paginilor (aplicatiile care stau in spate se mai numesc si daemon). Sarcinile acestui proces sunt:

- de a salva paginile pe disc;
- folosind o combinatie de algoritmi selecteaza urmatoarea pagina victima pentru inlocuire;
- decide cata memorie virtuala va fi alocata

Fenomenul de trashing

Acest fenomen apare atunci cand timpul alocat incarcarii sau scoaterii unei paginii din buffer este mai mare decat executia paginii. De exemplu atunci cand un proces are alocat un numar mai mic de cadre in memorie decat este necesar. Exista mai multe metode de evitare a trashingului (de exemplu folosirea algoritmilor globali sau locali de inlocuire a paginilor). Algoritmii globali presupun scoaterea din memorie a altor procese in favoarea procesului curent.



Analiza impactului erorilor de pagina: Pentru aceasta analiza se calculeaza timpul efectiv de accesare a memoriei. Se foloseste urmatoarea formula:

$$T_{\text{accesare}} = T_{\text{acces memorie}} * \text{probabilitatea pentru pageHit} + T_{\text{pageFault}} * \text{probabilitatea de pageMiss}$$

Evident raportul intre probabilitatea de page Hit si probabilitatea de page Miss este complementar → daca p este probabilitatea de page Miss atunci 1-p este probabilitatea de page Hit.

These are typical performance levels of a TLB: [\[47\]](#)

- size: 12 bits – 4,096 entries
- hit time: 0.5 – 1 clock cycle
- miss penalty: 10 – 100 clock cycles
- miss rate: 0.01 – 1% (20–40% for sparse/graph applications)

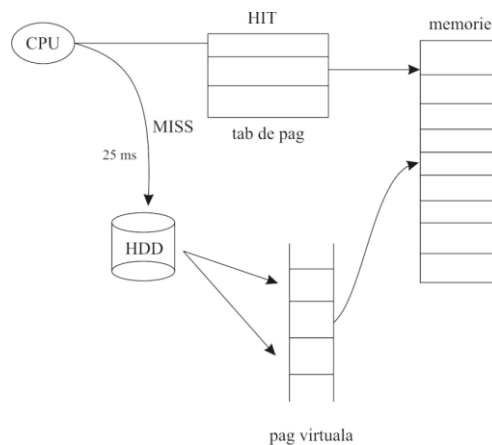
Exercitiu: If a TLB hit takes 1 clock cycle, a miss takes 30 clock cycles, and the miss rate is 1%, the effective memory cycle rate is an average of $1 \times 0.99 + (1 + 30) \times 0.01 = 1.30$ (1.30 clock cycles per memory access).

Exemplu: Se considera timpul de acces la memorie 20 ns, $T_{\text{accesHDD}}=25$ ms. Fie p probabilitatea de page fault. Sa se calculeze: care este probabilitatea de page fault necesara pentru ca timpul de accesare sa se incadreze in 5 procente din timp accesare memorie (adica 20 ns).

$$T_{\text{accesare}} = 20\text{ns} * (1-p) + 25\text{ms} * p = 21\text{ns}$$

$$21 = 20 * 10^{-9} (1-p) + 25 * 10^{-3} * p \rightarrow p \approx \dots$$

Acest proces este ilustrat in figura urmatoare:



Alocarea spatiului liber. Tipuri de alocatoare

1. **Alocatorul cu harti de resurse:** el foloseste un tabel in care sunt marcate drept ocupate sau libere diferite blocuri specificandu-se adresa de inceput si lungimea acestuia:

Adresa inceput	Lungime	Stare
00h	100 octeti	Libera
064h	50 octeti	Ocupata

Fiecare bloc contine dimensiunea blocului liber si un mecanism de calcul al pozitiei blocului urmator. Acest alocator parcurge crescator tabela pana gaseste un spatiu liber care va fi alocat. Acest alocator este simplu dar ineficient si performanta i se deterioreaza din pricina fragmentarii.

2. **Alocatorul cu puteri ale lui 2:** Pentru a imbunatati cautarea acest alocator foloseste blocuri de dimensiune puteri ale lui 2. Astfel daca o zona nu poate fi alocata se alocata automat o zona superioara urmatara putere a lui 2. Metoda se mai numeste si metoda injumatatirii deoarece ordinea de alocare a spatiilor se face prin dublarea spatiului.
3. **Alocatorul lui Fibonacci:** foloseste urmatoarea regula de alocare: $a_i = a_{i-1} + a_{i-2}$. Acest alocator are avantajul ca o zona eliberata intre alte 2 zone libere rezulta in unirea celor 3 zone datorita modului simplu de calcul al spatiului alocat.

CURS 13

Gestionarea sistemului de intrare-iesire

Toate calculatoarele au dispozitive fizice pentru acceptarea intrarilor si producerea iesirilor. Acestea se mai numesc si dispozitive periferice. Rolul sistemului de operare este de a gestiona aceste echipamente. Subsistemul de gestionare are multe parti comune deoarece anumite elemente software sunt independente de constructia dispozitivelor. Functiile sistemului de operare sunt:

- Generarea comenzilor catre dispozitiv
- Tratarea intreruperilor de intrare-iesire
- Tratarea erorilor
- Furnizarea unei interfete utilizator flexibile

Dificultatile in gestionarea echipamentelor periferice provin de la faptul ca exista diferite viteze de acces, diferite unitati de transfer si diferite tipuri de erori in functie de periferic.

Obiectivele gestiunii sistemului de intrare-iesire:

1. Independenta fata de periferice: programele sa se potriveasca pentru cat mai multe periferice
2. Independenta fata de codul de caractere
3. Eficienta operatiilor (ex: copiere de pe HDD pe stick)

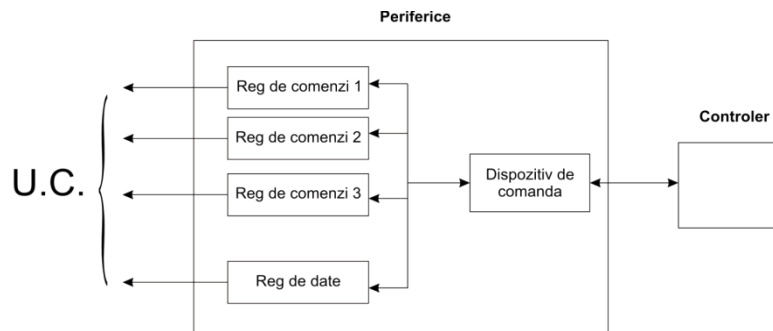
Clasificarea dispozitivelor periferice:

1. Din punct de vedere functional echipamentele periferice sunt:
 - De intrare-iesire
 - De stocare
2. Din punct de vedere al modului de operare:
 - Periferice dedicate: pot servi un singur proces la un moment dat
 - Periferice distribuite: pot servi mai multe procese la un moment dat (ex:HDD-ul)
3. Dupa modul de transfer si modul de memorare a informatiei
 - Periferice cu transfer de tip bloc: memorarea informatiei se face in blocuri
 - Periferice pe caracter: din aceasta categorie face parte tastatura
 - Periferice care nu sunt de tipurile anterioare: Ex. Ceasul real

Structura hard a echipamentelor periferice:

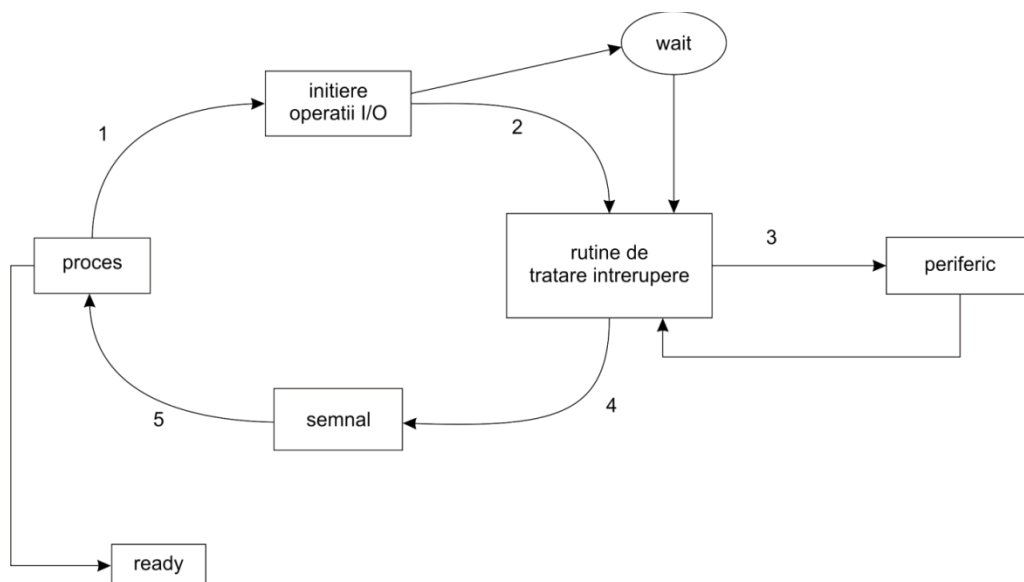
Exista 2 componente ale unui echipament periferic:

- Dispozitivul periferic
- Controlerul



Dispozitivul de comanda selecteaza intre registrii de comenzi.

Controlerul are rolul de a sintetiza comenzi pentru echipamentul periferic. Daca este o informatie de scriere informatia va fi stocata in registrul de date. Perifericul pe baza acestor comenzi executa operatiile si transmite catre controler rezultatul operatiei si eventualele erori. Controlerul este cel care preia aceste informatii si le transmite catre sistemul de operare. Din punct de vedere software lucrul cu intreruperi arata astfel:



Un proces care trebuie sa execute o operatie de intrare-iesire poate fi blocat in urma unei operatii **wait** la un semafor pana cand dispozitivul periferic devine disponibil. In acest moment se executa operatia de intrare-iesire si procesul este trezit printr-o operatie de primire a unui semnal. In acest moment procesul este deblocat si trece in starea READY.

Rutina de tratare a intreruperii (1)

1. Identificarea sursei de întreruperi (ce periferic a generat acea intrerupere).
2. Reinitializarea liniei de intreruperi
3. Memorarea starii echipamentului este necesara pentru cazul in care se executa mai multe operatii simultan. Fiecare accesare a dispozitivului periferic se face cu anumite caracteristici.
4. Trezire proces prin transmiterea unui semnal: „signal”

Componenta sistemului de operare *care depinde de dispozitivul periferic asociat* cu rolul de a transpune comenzile de nivel înalt ale sistemului de operare in comenzi la nivelul registrilor controlerului comenzile se numește **driver**.

Problema driverului este locatia in care acesta trebuie plasat. Exista 2 posibilitati:

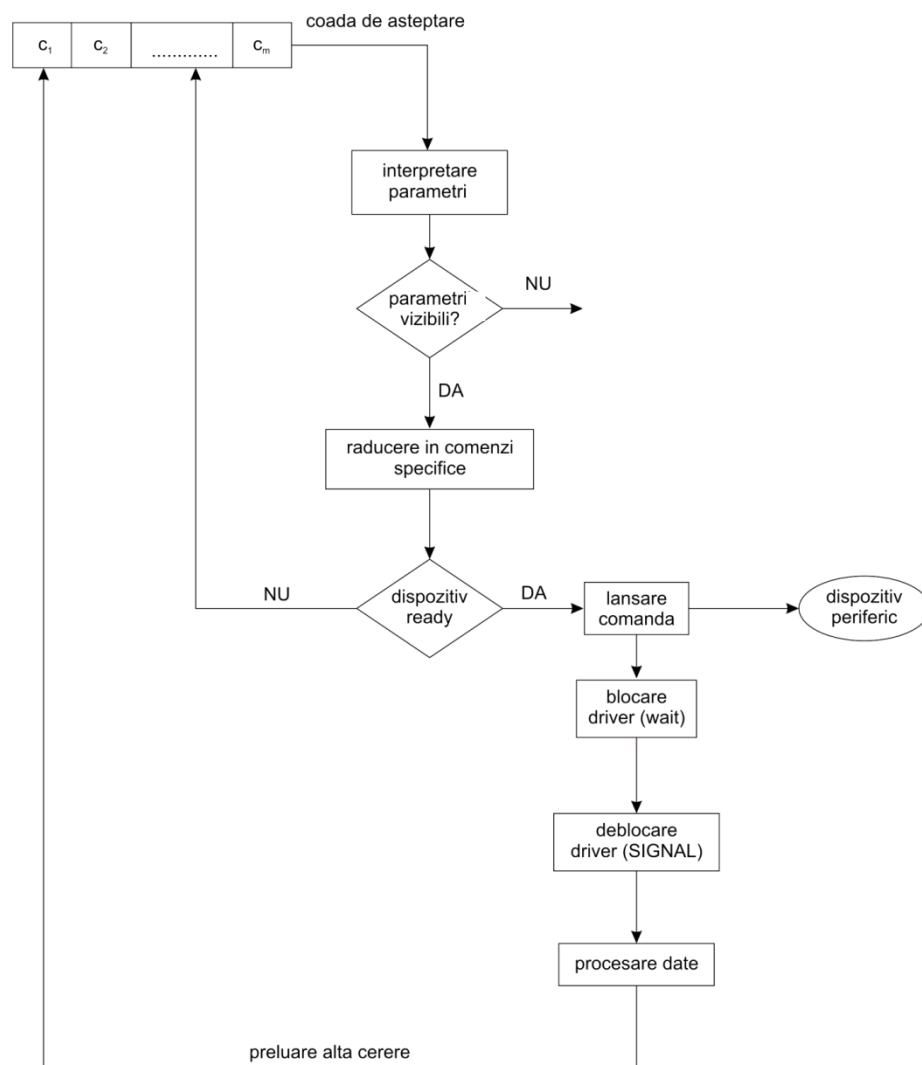
- In spatiul utilizator
- In kernel

De obicei driverul se plaseaza in kernel pentru a micsora timpul de acces si pentru o flexibilitate mai mare.



Sistemele de operare care implementează driverul in spatiul utilizator fac acest lucru pentru a separa executia driverului de executia sistemului de operare. Scopul este ca in cazul funcționarii eronate a driverului functiile sistemului de operare sa nu fie afectate.

Modul de funcționare al unui driver este urmatorul:



Etapa de procesare date implica citirea respectiv scrierea datelor catre dispozitivul periferic. Acestea sunt preluate in memoria calculatorului si pot genera diferite erori legate de disponibilitatea memoriei RAM a sistemului de calcul.

Interactiunea componentelor software pentru intrare-iesire:

UTILIZATOR

primitive de nivel utilizator
prog. sist. independente de dispozitiv
DRIVER
rutina de tratare a intreruperii

PERIFERIC

	Descriere
<i>Primitive de nivel utilizator:</i>	La acest nivel sunt plasate operatiile de I/O
<i>Program sistem independent de dispozitiv</i>	La acest nivel se efectueaza identificarea dispozitivului periferic
<i>Driverul</i>	are rolul de a prelua cererile din coada de asteptare, interpreteaza parametri acestora, executa operatii de wait (pt a bloca) si signal (pt a iesi) si trimite respectiv preia datele de la dispozitivul periferic.
<i>Rutina de tratare a intreruperii</i>	identifica starea dispozitivului si trimite un semnal catre driver pentru a semnaliza incheierea operatiei in dispozitivul periferic.

CURS 14

Sistemul de fisiere

Pentru aplicatiile software sunt necesare 2 elemente:

- Stocarea informatiei
- Regasirea informatiei

Procesorul studiat pana acum nu indeplineste aceste 2 conditii. Prin urmare se introduce entitatea numita fisier ca fiind o resursa logica a sistemului de operare pastrata intr-un mediu extern de memorare. Fisierul este unitatea logica de stocare a informatiei, este o colectie de informatii mapata la dispozitivul fizic; informatiile sunt definite de creatorul fisierului.

Operatii cu fisiere:

- o Creare
- o Citire: Read
- o Scriere: Write
- o Denumire
- o Asigurarea mecanismelor de acces concurent
- o Asigurarea persistentei datelor

Clasificarea fisierelor:

1. Dupa structura:
 - Fisier tip secventa de octeti a caror interpretare o fac programele utilizator
 - Fisere cu structura arborescenta specifica calculatoarelor cu spatiu de stocare de mare capacitate
 - Fisiere tip secventa de inregistrari specific primelor echipamente periferice (ex. fisiere cu o dimensiune maxima a liniei dedicata imprimantei)
2. Dupa tip:
 - Fisiere normale de tip text sau binar
 - Directoare care permit organizarea fisierelor
 - Fisiere speciale (fisierul FIFO sau asa numitele pipe cu nume)
3. Dupa acces:
 - Fisiere cu acces secvential
 - Fisiere cu acces direct care permit atat scrieri si citiri
 - Fisiere cu acces indexat

Operatii cu fisiere:

- o Creare fisier
- o Citire: Read
- o Scriere: Write
- o Stergere: Unlink
- o Inchiderea fisierului: Close
- o Cautare in fisier: Seek
- o Redenumirea fisierelor: Rename
- o Setarea atributelor proprietatilor unui fisier: **chmod**

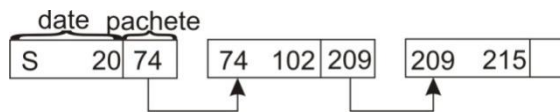
Implementarea sistemului de fisiere:

Un sistem de operare lucreaza la nivel logic cu **blocuri formate din unul sau mai multe sectoare fizice**. Modul de alocare al blocurilor pentru fisiere are urmatoarele posibilitati:

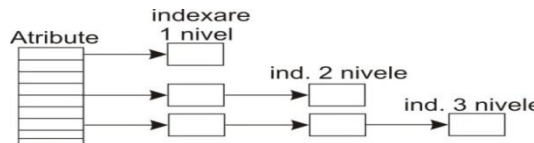
- **Alocare contigua:** are **avantajul** cunoasterii rapide a dimensiunii prin determinarea blocului de inceput si blocului de sfarsit; **Dezavantajele** sunt: ca fisierele nu pot creste si ca in functie de dimensiunea blocurilor o mare parte a spatiilor este pierduta



- **Alocarea inlantuita:** fisierul este o lista inlantuita de blocuri pe disc. **Avantaje:** fisierele pot creste; blocurile pot fi imprastiate oriunde pe hdd si accesul secvential se face usor. Pentru a cunoaste inlantuirea se folosea un pointer atasat zonei de date pentru a identifica locatia urmatoare. Aceasta structura este folosita de sistemul de fisiere FAT (file allocation table). **Dezavantaje:** deteriorarea pointerului implica pierderea partilor din fisier; accesul se ingreuneaza odata cu inmultirea numarului de legaturi.



- **Alocarea indexata:** este folosita in sistemele de operare unix, linux si pune la un loc toti pointerii intr-un bloc de index. **Avantajele:** sunt ca dispare fragmentarea externa; avantajos in cadrul multor fisiere de dimensiune mica. **Dezavantajul** este costul suplimentar de accesare al blocului de index. Un nod de indexare UNIX contine un camp de date care stocheaza attributele fisierului si o legatura spre un alt bloc. Adresele indexate unde se gasesc informatiile din fisier permit o indirectare pana la 3 niveluri.



- **alocare cu jurnalizare:** in sistemul de alocare anterior folosit de UNIX, pentru a sterge un fisier trebuiesc parcurse 2 etape:
 - o stergerea inregistrarii din director
 - o marcarea spatiului si a nodului libere

Daca are loc o intrerupere a alimentarii intre cele 2 etape este posibil sa ramana locatii in blocul de index ca referinte la fisiere neexistente. Rezultatul este pierderea de date. Sistemul cu jurnalizare folosit de NTFS (new technology file system) este utilizat de windows XP, Vista, 7. Sistemul cu jurnalizare ofera suport pentru metadata, securizare acces prin liste de control si un sistem de fisiere care noteaza intr-un log operatiile inainte de a fi executate. Daca se intrerupe alimentarea cu tensiune a sistemului de calcul, operatiile pot fi completate pe baza analizei listei cu locurile pastrate. Acest sistem de fisiere intr-o versiune mai veche era utilizat si de sistemul de operare IBM OS2 pentru ca a fost realizat de Microsoft in colaborare cu IBM.

Drepturi de acces la fisier:

Sistemul de operare Unix foloseste un sistem eficient de pastrare a drepturilor de acces (user, group si others) separand drepturile pentru grupurile de utilizatori si altii. Operatiile restrictionate sunt citire, scriere si

executie. In sistemul Windows modalitatea de control a accesului o reprezinta listele de acces continand drepturile pentru utilizatori sau grupurile de utilizatori. La accesarea unui fisier se verifica lista de acces asociata.

FAT12, FAT16 & FAT32

- File allocation table = "FAT".
- Major variants = FAT12, FAT16 & FAT32.
- Each has an increasing number of clusters, and maximum file and volume sizes.
- FAT32 remains popular due to wide OS compatibility.
- Still widely used for removable media.

FAT12, FAT16 & FAT32

	Max file size	Max volume size
FAT12	32MB (8KB clusters)*	32MB (8KB clusters)*
FAT16	2GB/4GB**	16GB (256KB clusters)**
FAT32	4GB	32GB (Windows format) 2TB (other OS) 16TB (theoretical)

*16MB with 4KB clusters

**Different FAT16 versions were limited to a maximum volume size of 2GB, 4GB or volume size, with volumes limited to 2GB, 4GB, 8GB or 16GB depending on cluster & sector size.

Journaling = mentine o lista a modificarilor pentru a putea reveni in cau unei problem/crash

NTFS

- New technology file system = "NTFS".
- File size limit of 16 exabytes (EB).
- Journaling file system (to avoid corruption).
- Supports file permissions and encryption.
- Windows must be installed on NTFS drive.
- Limited non-Windows OS compatibility — eg read-only in macOS and older Linux distros.

exFAT

- Extended file allocation table = "exFAT".
- File system optimized for high-capacity USB flash drives and memory cards.
- Maximum file size = 16EB.
- Default file system for SDXC cards.
- Broader non-Windows OS support than NTFS — including read and write on macOS.

ext2, ext3 & ext4

- In 1992, the "extended file system" (ext) was launched for Linux.
- In 1993, ext2 was released.
- In 2001, ext3 introduced journaling.
- In 2008, ext4 became the Linux default FS.
- Files up to 16TB and volumes up to 1EB.
- No native Windows or macOS support.

HFS, HFS+ & APFS

- HFS = hierarchical file system for macOS.
- HFS also known as "Mac OS Standard".
- In 1998, HSF+ ("HSF Extended" or "Mac OS Extended") introduced with journaling.
- Files and volumes up to 8EB (macOS 10.4).
- In 2017, Apple file system (APFS) launched.
- No native Windows or Linux support.

ZFS

- ZFS = zed file system.
- Created by Sun Microsystems, and now developed by the OpenZFS Project.
- Integrated volume manager to control storage hardware.
- Hence provides increased data protection.
- Available for Linux, FreeBSD and TrueOS.

Choosing a File System

- For a system drive, use NTFS (Windows), ext4 (Linux) or HFS+/APFS (macOS).
- For USB drives and memory cards, use FAT32 for capacities under 32GB . . .
- . . . and use exFAT for 32GB+ or 4GB+ files.
- For other drives, use NTFS if Windows based, exFAT if PC/Mac based, or FAT32.

Securitatea reprezintă abilitatea de a evita neplăcerile produse de orice risc, amenințare sau pericol. În practică, acest lucru este imposibil de realizat.

Incident de securitate este un eveniment apărut în cadrul rețelei, provenind din interiorul ori exteriorul rețelei, cu implicații asupra securității unui calculator sau a rețelei.

Se poate introduce un limbaj comun pentru descrierea incidentelor:

*Principalul obiect de studiu: **eveniment** (incident legat de securitate)*

- constă dintr-o **acțiune** executată asupra unei **ținte**
- acțiunea poate fi executată cu o **unealta**
- exploatând un anumit tip de **vulnerabilitate**
- cu un anumit **rezultat** (în mod normal neautorizat)

Abordarea problemei securității datelor într-o rețea presupune în primul rând identificarea cerințelor de funcționare pentru acea rețea, apoi identificarea tuturor amenințărilor posibile (împotriva cărora este necesară protecția). Aceasta analiză constă în principal în trei sub-etape:

- *analiza vulnerabilităților* - identificarea elementelor potențial slabe ale rețelei
- *evaluarea amenințărilor* - determinarea problemelor care pot apărea datorată elementelor slabe ale rețelei și modurile în care aceste probleme interferează cu cerințele de funcționare
- *analiza riscurilor* - posibilele consecințe pe care problemele le pot crea

Următoarea etapă constă în definirea politicii de securitate, ceea ce înseamnă să se decidă:

- care amenințări trebuie eliminate și care se pot tolera
- care resurse trebuie protejate și la ce nivel
- cu ce mijloace poate fi implementată securitatea
- care este prețul (financiar, uman, social etc.) măsurilor de securitate care poate fi acceptat

O rețea LAN are un singur administrator și o singură politică de securitate, în vreme ce rețelele WAN au mai mulți administratori și politici de securitate multiple.

Odată stabilite obiectivele politicii de securitate, următoarea etapă constă în selecția serviciilor de securitate - funcțiile individuale care sporesc securitatea rețelei. Fiecare serviciu poate fi implementat prin metode (mecanisme de securitate) variate pentru care sunt necesare așa-numitele funcții de gestiune a securității. Gestiunea securității într-o rețea constă în controlul și distribuția informațiilor către toate sistemele deschise ce compun acea rețea în scopul utilizării serviciilor și mecanismelor de securitate și al raportării evenimentelor de securitate ce pot apărea către administratorii de rețea.

Aspecte ale securității în rețele de calculatoare:

– **Identitatea**: va cuprinde elementele de **autentificare** și **autorizare** la nivelul rețelei.

– Autenticitatea: presupune ca două entități aflate într-un schimb de mesaje se pot identifica una pe cealaltă. În prima fază, la inițierea conexiunii, acest serviciu asigură că cele două entități sunt autentice. În al doilea rând, autenticitatea presupune că transferul de date dintre cele două entități nu este interferat astfel încât o a treia entitate poate să se legitimeze ca fiind una din ele.

– Autorizarea: este abilitatea de a limita și controla accesul în rețea (la sisteme sau aplicații). Pentru a realiza acest serviciu, fiecare entitate care încearcă să aibă acces trebuie mai întâi identificată și apoi verificate drepturile de acces în sistem.

– **Integritatea**: este o componentă a securității care cuprinde infrastructura de securitate (accesul fizic și logic) precum și securizarea perimetrului. Ea se referă la asigurarea consistenței informațiilor, încrederea în date sau resurse (în cazul transmiterii unui mesaj prin rețea, integritatea se referă la protecția împotriva unor tentative de falsificare a mesajului);

– **Confidențialitatea**: asigura faptul că transmisiile de date de-a lungul rețelei au caracter privat. Există numeroase posibilități pentru confidențialitatea informațiilor de la protecție fizică până la algoritmi matematici.

– **Disponibilitatea**: va asigura faptul că toate resursele rețelei sunt disponibile personalului sau proceselor autorizate.

– **Nerepudierea**: măsură prin care se asigură faptul că, după emiterea/recepționarea unei informații într-un sistem de comunicații securizat, expeditorul/destinatarul nu poate nega, în mod fals, că a expedit/primit informații. *Se previne ca nici o entitate să nu refuze să recunoască un serviciu executat. Când un mesaj este trimis, destinatarul poate demonstra că mesajul primit este cel trimis de emițător. Similar, când un mesaj este primit, emițătorul poate demonstra că mesajul primit este cel primit de destinatar.*

– **Auditul**: este necesar pentru monitorizarea și verificarea securității la nivelul firmei

1. Vulnerabilități în rețele de calculatoare

Vulnerabilitatea este o slăbiciune a unui sistem hard/soft ce permite utilizatorilor neautorizați să aibă acces asupra sa. Nici un sistem nu este integral sigur. De multe ori vulnerabilitățile apar/sunt facilitate și datorită proastei administrări.

Tipuri de vulnerabilități: – permiterea refuzului serviciilor (DOS – Denial Of Service); – permiterea utilizatorilor locali cu privilegii limitate să-și mărească aceste privilegii fără autorizație;

– permiterea utilizatorilor externi să acceseze rețeaua/sistemul local în mod neautorizat;

Cauzele posibile ale existenței vulnerabilităților (90% sunt datorate utilizatorilor): – Erori existente în programe introduse deseori neintenționat – Ignorarea/nedocumentarea erorilor existente – Configurarea necorespunzătoare a programelor, serverelor și rețelelor – Lipsa suportului din partea producătorilor (e.g. rezolvarea greoaie a erorilor) – Comoditatea sau necunoașterea problemelor de securitate de către administrator ori de conducerea organizației.

2. Atacuri în rețele de calculatoare

Atac este un eveniment potențial distrugător provocat intenționat de persoane răuvoitoare.

Atribute ce trebuie considerate pentru a estima reușita unui atac (cunoașterea profilului atacatorului): – Resursele disponibile (financiare, tehnice,... + pregătirea în domeniu) – Timpul alocat (atacatorii răbdători vor avea mai mult succes) – Riscul asumat – depinde de obiective (atacul ar putea fi revendicat sau nu de cracker) – Accesul la Internet și calitatea acestuia: tip (dial-up, conexiune satelit,...), mod de alocare a adreselor IP etc. – Obiectivele urmărite (recunoaștere mondială, denigrarea țintei, furt de informații sau bani etc.)

Niveluri de atac – **Oportunist**: Atacul are un scop “recreational” și nu are obiective/ținte clar definite. Se utilizează programe disponibile liber pentru a scana sau testa vulnerabilități uzuale. Nu necesită acces în interiorul sistemului. Cunoștințe vagi despre sistemul/organizația țintă • Măsurile de precauție: – ziduri de protecție (firewall-uri) – actualizarea versiunilor de programe

– **Intermediar** Obiectiv este conturat, la nivelul organizației. Se efectuează aceleași acțiuni ca la atacul “recreational”, dar se încearcă ascunderea lor. Atacatorul are mai multă răbdare decât în cazul unui atac oportunistic. Cunoștințe tehnice mai profunde. Probabilitate mai mare de succes, posibil efecte mai puternice

– **Sofisticat:**

- Obiectiv foarte bine conturat. Ținta este de cele mai multe ori o organizație. Atacurile pot trece peste măsurile de prevedere
- Atacatorul va avea multă răbdare. Se investește timp pentru adunarea de informații despre sistemul/organizația țintă
- Necesită foarte bune abilități tehnice și are o probabilitate mare de succes.

Tipuri de atac – Accesul utilizator

- Atac prin acces via utilizator obișnuit sau cu privilegii superioare

• Etape:

- Colectarea de informații – utilizatori, vulnerabilități,...
- Exploatarea
- Deteriorarea
- » Modificare de informații
- » Acces la date importante
- » Asigurarea 99% accesului ulterior la sistem
- » Modificarea jurnalelor de sistem

– **Accesul de la distanță la servicii-retea**

- Nu necesită acces utilizator la sistem
- Creează refuzuri de servicii prin cereri incorecte, eventual cu “caderea” serviciilor prost proiectate
- Etape:
- Colectarea de informații – identificarea de servicii
- Exploatarea – trimiterea de pachete la portul găsit
- Deteriorarea
- » Distrugerea unui serviciu de rețea
- » Defectarea/incetinirea (temporară) a unui serviciu sau a sistemului

– **Accesul de la distanță la diverse aplicații**

- Trimitere de date invalide aplicațiilor, nu serviciilor de rețea (traficul nu e afectat)
- Nu necesită obținerea de acces utilizator
- Etape:
- Colectarea de informații – identificarea aplicației (e.g. server sau client Web, aplicație gen MS Office)
- Exploatarea – trimiterea conținutului, direct sau indirect (i.e. via e-mail), spre aplicație
- Deteriorarea
- » Stergerea/copierea fișierelor utilizatorilor
- » Modificarea fișierelor de configurație

Ținta atacurilor în rețele de calculatoare

– Organizații publice sau guvernamentale

- Recunoaștere în rândul cracker-ilor
- Captarea atenției mass-media
- Revendicări etice, politice,...
- Furnizori de servicii Internet
- Sabotarea activității
- Companii private
- Discreditare
- Furt de informații
- Răzbunare din partea foștilor angajați

☆ First Mac Botnet Activated, Engages in DDoS Attacks

posted by [Thom Holwerda](#) on Sat 18th Apr 2009 09:27 UTC



Remember the Mac trojan that we [reported about earlier this year](#)? A trojan was found piggybacking on the back of copies of iWork and Photoshop CS4 found on warez sites and networks, and it would install itself after the user had entered his or her administrator password during the software's installation. This trojan didn't seem like much of a threat back then, but as it turns out, [it's now in use in the first Macintosh botnet](#).

Security researchers from Symantec [have found evidence](#) that said trojans, OSX.Iservice and OSX.Iservice.B, are being used in creating a botnet used for DDoS attacks. There's at least [one documented case of these trojans being used for DDoS attacks](#), and the researchers have found out that the botnet has encryption, a peer-to-peer engine, and remote startup capabilities.

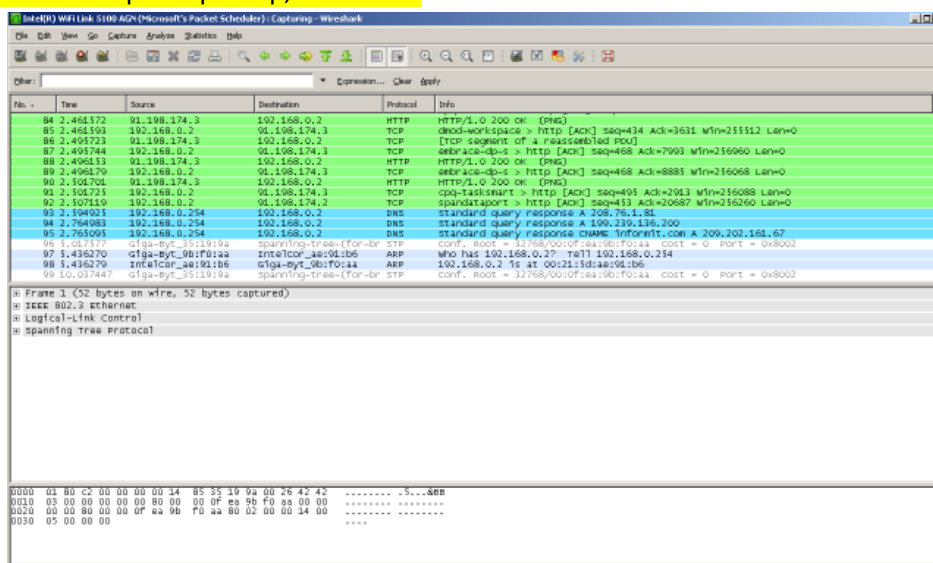
Persoane fizice

- Cu scop recreațional

Moduri de atac – Bomba e-mail (e-mail bombing)

- Trimiterea repetată a unui mesaj (de dimensiuni mari) spre o adresă e-mail a unui utilizator
- Încetinește traficul, umple discul
- Unele atacuri pot folosi adrese e-mail multiple existente pe serverul țintă
- Se poate combina cu falsificarea adresei (e-mail spoofing)
- Spam (e-mail spamming)
- Trimiterea de mesaje nesolicitate (reclame)
- Adresa expeditorului e falsă
- Efectul atacului e accentuat dacă mesajul este trimis pe o listă de discuții
- Abonarea la liste de discuții
- “Atac” ce determină enervarea victimei, facilitat de diverse programe disponibile în Internet
- Cauzează trafic inutil de rețea
- Falsificarea adresei expeditorului (e-mail spoofing)
- Folosită pentru ascunderea identității expeditorului sau pentru determinarea utilizatorului să răspundă la atac ori să divulge informații (e.g. parole)
- Slăbiciunea e datorată modului în care operează protocolul SMTP
- Utilizatorii trebuie educați să nu răspundă expeditorilor necunoscuți și să nu divulge informații

confidentiale – *Refuzul serviciilor (Denial Of Service)* • Degradează calitatea funcționării unor servicii sau conduce la dezafectarea lor • Bombardament cu pachete (packet flood) – se trimite un număr mare de pachete spre o anumită gazda de la o singură sursă ori provenind de la surse multiple (**Distributed DOS**) – Segmente TCP (cu setarea SYN, ACK sau RST) – Pachete ICMP (ping flood) – Pachete UDP • Se poate falsifica adresa IP a expeditorului (IP spoofing) • Se pot modifica porturile sursă/destinație (pentru a trece de firewall-uri) • Exemple – SYN flood – cereri multiple de realizare de conexiuni – Ping of death – atac cu pachete ICMP mari – Teardrop – exploatarea implementărilor TCP/IP care nu gestionează corect pachetele IP – Smurf – atac ICMP asupra adresei de broadcast – *Depasirea capacității bufferelor (buffer overflow)* • Unele programe pot alocă spațiu insuficient pentru unele date, depășirile survenite pot produce executarea de comenzi ca root • Uzual, atacul provine din interior, dar poate fi și din exterior (via un cal troian) – *Interceptarea rețelei (IP sniffing)* • Monitorizarea datelor care circula printr-o interfață de rețea – se pot detecta parole transmise necriptate • Atacul provine din interior • Pentru rețele de viteză mare (100 M/s) unele pachete nu pot fi captate de sniffer • Soft-ul de interceptare trebuie supravegheat • Exemple: tcpdump, Wireshark



– *Cai troieni (trojan horses)* • Programe rău intenționate, “deghizate” sub forma unor executabile utile • Apelează programe neautorizate sau sunt modificate, incluzând cod nelegitim • Acțiuni: colectarea de informații, distrugerea de informații, lansarea de atacuri spre alte sisteme • Exemple: sendmail sau “vaduva neagră” (blochează sau corupe browsere Web) – *Usi ascunse (back doors / traps)* • Caz particular de cai troieni • Creează o “poartă” (e.g. utilizator, port,...) care permite accesul ulterior la calculator și/sau câștigarea de privilegii – *Viermi (worms)* • Programe care se multiplică, transferându-se pe alte calculatoare și efectuând distrugeri. Exemplu: Internet Worm (1988) – *Ghicirea parolelor (password guessing)* • Folosirea unui program ce determină parolele prost alese (prea simple) – Prea scurte, utilizează cuvinte de dicționar, numerice • Protecție prin /etc/shadow, reguli stricte de schimbare a parolelor, educarea utilizatorilor, folosirea de programe de tip spărgător de parole (password cracker) – *Virusi* • Programe ce efectuează operații nedorite (distructive), cu capacități de “multiplicare”. Ele realizează infectarea altor programe (uzual, executabile) • Mai puțin răspândiți în Unix/Linux, de obicei având efect doar dacă se execută sub auspiciul de root • Pot genera și e-mail bombing • Remedii: utilizarea de antiviruri și porți de e-mail

3. Prevenirea atacurilor

Modelul de securitate pentru un sistem (un calculator sau o retea de calculatoare) poate fi vazut ca având mai multe straturi ce reprezintă nivelurile de securitate ce inconjoara subiectul ce trebuie protejat. Fiecare nivel izoleaza subiectul si il face mai dificil de accesat in alt mod decat cel in care a fost prevazut.

1. *Securitatea fizica* reprezinta nivelul exterior al modelului de securitate si consta, in general, in incuierea echipamentelor informatice intr-un birou sau intr-o alta incinta precum si asigurarea pazei si a controlului accesului. Aceasta securitate fizica merita o consideratie speciala. Una dintre problemele mari o constituie salvarile sub forma de copii de rezerva (backup) ale datelor si programelor, precum si siguranta pastrarii suportilor de salvare. Retelele locale sunt, in acest caz, de mare ajutor, copiile de rezerva putandu-se face prin retea pe o singura masina ce poate fi mai usor securizata. O alta problema importanta in securitatea unui sistem informatic o constituie pur si simplu sustragerile de echipamente. In plus, celelalte masuri de securitate (parole etc.) devin nesemnificative in cazul accesului fizic neautorizat la echipamente.
2. *Securitatea logica* consta din acele metode logice (software) care asigura controlul accesului la resursele si serviciile sistemului. Ea are, la randul ei, mai multe niveluri impartite in doua grupe mari : *niveluri de securitate a accesului si niveluri de securitate a serviciilor*.
 - *Securitatea accesului* cuprinde:
 - accesul la sistem, care este raspunzator de a determina daca si cand este reseaua accesibila utilizatorilor si in ce conditii. El poate fi raspunzator de asemenea si de gestionarea evidentei accesului. Accesul la sistem poate efectua si deconectarea fortata in anumite cazuri (ex. expirarea contului, ora de varf, ...)
 - accesul la cont care verifica daca utilizatorul ce incearca sa se conecteze are un nume si o parola valida.
 - drepturile de acces (la fisiere, resurse, servicii etc.) care determina de ce privilegii dispune un utilizator (sau un grup de utilizatori) dat.
 - *Securitatea serviciilor* (care se afla "sub" securitatea accesului) controleaza accesul la serviciile unui sistem (masina, retea). Din acest nivel fac parte:
 - *controlul serviciilor* care este responsabil cu functiile de avertizare si de raportare a starii serviciilor, precum si de activarea si dezactivarea diverselor servicii oferite de catre sistemul respectiv
 - *drepturile la servicii* care determina exact cum foloseste un anumit cont un serviciu dat (acces la fisiere, resurse, prioritate,...)

Elaborarea de politici de securitate implica: – Planificarea cerințelor de securitate • Confidențialitate, integritate, disponibilitate, control – Evidențierea riscurilor – Analiza raportului cost-beneficii • Costurile prevenirii, refacerii după dezastru etc. – Stabilirea politicilor de securitate • Politica generala (naționala, organizaționala,...) • Politici separate pentru diverse domenii protejate • Standarde & reglementari (recomandări) • Masurile luate pot fi tehnice si non-tehnice

Elaborarea de politici de securitate – exemplu:

- Gestionarea accesului (nume de cont, alegerea si modul de schimbare a parolelor, blocarea terminalului, politica de acces din exterior,...)
- Clasificarea utilizatorilor (grupuri, permisiuni, utilizatori speciali, utilizatori administratori)
- Accesul la resurse (drepturi de acces la fisiere, directoare, criptarea fișierelor importante,...)
- Monitorizarea activității (fișiere de jurnalizare)
- Administrarea copiilor de siguranță (tipuri de salvări, medii de stocare, durata păstrării,...)

Supraviețuirea reprezintă capacitatea unui sistem (calculator/rețea) de a-si îndeplini misiunea, in timp util, in prezenta *atacurilor, defectelor sau accidentelor*

- Atac =eveniment potențial distrugător provocat intenționat de persoane răuvoitoare • Defect =eveniment potențial distrugător cauzat de deficiente ale sistemului sau ale unui factor de care depinde sistemul (e.g. defecte hard, bug-uri soft, erori ale utilizatorilor) • Accident =evenimente aleatoare (neprevăzute); exemple: dezastre naturale, căderi de tensiune
- Sistemul trebuie sa-si duca pana la capăt misiunea chiar daca unele componente sau părți din sistem sunt afectate ori scoase din uz • Sistemul trebuie sa susțină măcar îndeplinirea funcțiilor vitale – Identificarea serviciilor esențiale • Proprietati ale sistemului: – Rezistenta la atacuri – Recunoașterea atacurilor si efectelor lor – Adaptarea la atacuri • Instrumente sub Linux (Unix): – Utilitare de rețea: ping, traceroute, netstat, ifconfig, route, host, finger, telnet – Scanere de porturi: NMAP – Interceptoare de rețea: tcpdump, wireshark – Testarea securității locale: /etc/shadow, Crack, Titan – Verificări asupra sist. de fisiere: tripwire, showmount – Salvări de siguranta: tar, dump, amanda – Verificarea daemonilor: chkconfig – Protecția TCP/IP: iptables (firewall), activarea mecanismului SYN cookies in nucleu

Actiuni pentru securizare SO

- Siguranța fizică a sistemului - Instalarea mașinii trebuie realizată într-un loc sigur, să nu fie expusă contactului cu persoane neautorizate. Acestea nu trebuie să aibă posibilitatea sau timpul necesar de a înlătura carcasa, de a modifica configurația hardware, de a opri și apo reporni mașina (eventual în modul single), de a înlocui sau copia informațiile discuri sau de a inocula programe răuvoitoare (cai troieni). De asemenea, mediile de stocare a salvărilor de siguranță trebuie să fie păstrate într-un loc închis, fără posibilitate de acces (e.g., un seif).
- Salvările de siguranță - Se recomandă salvarea periodică cel puțin a fișierelor importante și, dacă este posibil a întregului conținut al sistemelor de fișiere.
- Drepturile de acces la fișierele importante - Trebuie acordată o atenție sporită drepturilor de acces la fișierele importante: fișierele de configurare ale diverselor servicii instalate în sistem, fișierele jurnal (log-uri), executabilele care nu trebuie să poată fi apelate de către utilizatorii obișnuiți, precum și alte fișiere importante (spre exemplu, baze de date MySQL, PostgreSQL etc.), executabilele și scripturile de inițializare ale sistemului nu trebuie să poată fi modificate decât de root / administrator.
- Execuția daemonilor/serviciilor/proceselor - Se recomandă ca aplicatiile utilizate curent să ruleze pe sistem. Mai mulți(te) daemone / servicii înseamnă o încărcare mai mare a sistemului, precum și un nivel de vulnerabilitate mai mare. De asemenea, o mare parte a daemonilor / serviciilor (care oferă diverse servicii) nu trebuie executați sub root / administrator, ci sub utilizatorii speciali (de exemplu, daemonul HTTP rulează sub utilizatorul www). • Scripturile CGI - Scripturile CGI nu trebuie executate ca root. Acestea trebuie plasate într-un singur director, în care nu se va permite accesul utilizatorilor, iar modificările asupra scripturilor trebuie monitorizate.
- Porturile - Anumite servicii pot fi accesate prin rețea, de pe alte mașini. Pentru aceasta, ele așteaptă conexiuni pe anumite porturi (e.g., serverul HTTP pe portul 80). Aceste porturi pot constitui puncte vulnerabile ale sistemului (datorită vulnerabilităților care pot exista în aceste programe), putând fi detectate de la distanță cu ajutorul scanerelor. Aceste porturi trebuie protejate fie prin configurarea respectivelor servicii să accepte conexiuni doar de pe o anumită interfață de rețea, considerată sigură (e.g., rețeaua locală), fie prin configurarea unui firewall care să nu permită accesarea din exterior a porturilor în cauză.

- Accesul utilizatorului root / administrator în sistem - Din principiu, nu se recomandă permiterea accesului cu root / administrator decât de la consolele sistemului. Accesul de la distanță (cu SSH) va fi făcut cu un utilizator obișnuit, iar apoi va fi folosită comanda su. Sistemul FreeBSD nu permite accesul la distanță prin SSH folosind autentificarea ca root și, de asemenea, nu permite su decât din contul utilizatorilor ce aparțin grupului wheel.